



DWS层实现

从DWS层开始以主题划分进行统计。

按照主题划分的日统计宽表，基于DWB上的基础数据，整合汇总成分析某一个主题域的服务数据。

由于DWB实现了宽表，通过把相关字段放在同一张表中，可以大大减轻DWS数据计算时的消耗，提高计算效率问题。

1. 销售主题日统计宽表

1.1 建表

[..\第1章 项目介绍与环境准备\报表需求文档.xlsx](#)

可分析的主要指标有：销售收入、平台收入、配送成交额、小程序成交额、安卓APP成交额、苹果APP成交额、PC商城成交额、订单量、参评单量、差评单量、配送单量、退款单量、小程序订单量、安卓APP订单量、苹果APP订单量、PC商城订单量。

维度有：日期、城市、商圈、店铺、品牌、大类、中类、小类。

```
create database yp_dws;

DROP TABLE IF EXISTS yp_dws.dws_sale_daycount;

CREATE TABLE yp_dws.dws_sale_daycount (
  --dt STRING,
  province_id string COMMENT '省份id',
  province_name string COMMENT '省份名称',
  city_id string COMMENT '城市id',
  city_name string COMMENT '城市名称',
  trade_area_id string COMMENT '商圈id',
  trade_area_name string COMMENT '商圈名称',
  store_id string COMMENT '店铺id',
  store_name string COMMENT '店铺名称',
  brand_id string COMMENT '品牌id',
  brand_name string COMMENT '品牌名称',
  max_class_id string COMMENT '商品大类id',
  max_class_name string COMMENT '大类名称',
```





```
mid_class_id string COMMENT '中类id',
mid_class_name string COMMENT '中类名称',
min_class_id string COMMENT '小类id',
min_class_name string COMMENT '小类名称',
group_type string COMMENT '分组类型:store,trade_area,city,brand,min_class,mid_class,max_class,all',
-- =====日统计=====
-- 销售收入
sale_amt DECIMAL(38,2) COMMENT '销售收入',
-- 平台收入
plat_amt DECIMAL(38,2) COMMENT '平台收入',
-- 配送成交额
deliver_sale_amt DECIMAL(38,2) COMMENT '配送成交额',
-- 小程序成交额
mini_app_sale_amt DECIMAL(38,2) COMMENT '小程序成交额',
-- 安卓APP成交额
android_sale_amt DECIMAL(38,2) COMMENT '安卓APP成交额',
-- 苹果APP成交额
ios_sale_amt DECIMAL(38,2) COMMENT '苹果APP成交额',
-- PC商城成交额
pcweb_sale_amt DECIMAL(38,2) COMMENT 'PC商城成交额',
-- 成交单量
order_cnt BIGINT COMMENT '成交单量',
-- 参评单量
eva_order_cnt BIGINT COMMENT '参评单量comment=>cmt',
-- 差评单量
bad_eva_order_cnt BIGINT COMMENT '差评单量negative-comment=>ncmt',
-- 配送成交单量
deliver_order_cnt BIGINT COMMENT '配送单量',
-- 退款单量
refund_order_cnt BIGINT COMMENT '退款单量',
-- 小程序成交单量
miniapp_order_cnt BIGINT COMMENT '小程序成交单量',
-- 安卓APP订单量
android_order_cnt BIGINT COMMENT '安卓APP订单量',
-- 苹果APP订单量
ios_order_cnt BIGINT COMMENT '苹果APP订单量',
-- PC商城成交单量
pcweb_order_cnt BIGINT COMMENT 'PC商城成交单量'
)
COMMENT '销售主题日统计宽表'
PARTITIONED BY(dt STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```





1.2 导入数据

1.2.1 常用窗口函数

1.2.1.1 ROW_NUMBER

根据窗口分区中行的顺序，为每一行返回一个唯一的顺序号（从1开始）。

它不能用做于学生成绩的排名，一般多用于分页查询，比如求TopN。

Demo1. 按照订单金额排序获取序号

```
select
  order_amount,
  ROW_NUMBER() OVER(order by order_amount desc) rn
from yp_dwb.dwb_order_detail;
```

	order_amount	rn
1	198.00	1
2	165.60	2
3	149.40	3
4	138.00	4
5	134.52	5
6	126.70	6
7	126.70	7
8	126.70	8
9	126.70	9
10	126.70	10
11	126.52	11
12	126.52	12

Demo2. 获取金额排行Top10的订单

```
select *
from (
  select order_amount,
    ROW_NUMBER() OVER(order by order_amount desc) rn
  from yp_dwb.dwb_order_detail
)
where rn <=10;
```



	order_amount	rn
1	198.00	1
2	165.60	2
3	149.40	3
4	138.00	4
5	134.52	5
6	126.70	6
7	126.70	7
8	126.70	8
9	126.70	9
10	126.70	10

Demo3. 获取每个店铺金额排行Top10的订单

```
select *
from (
    select order_amount, store_id,
           ROW_NUMBER() OVER(partition by store_id order by order_amount desc) rn
    from yp_dwb.dwb_order_detail
)
where rn <=10
;
```

	order_amount	store_id	rn
1	3.00	419fc3243aff11e998ec7cd30ad32e2e	1
2	3.00	419fc3243aff11e998ec7cd30ad32e2e	2
3	3.00	419fc3243aff11e998ec7cd30ad32e2e	3
4	3.00	419fc3243aff11e998ec7cd30ad32e2e	4
5	10.00	315e2e923a7011e998ec7cd30ad32e2e	1
6	4.80	315e2e923a7011e998ec7cd30ad32e2e	2
7	4.00	d311268343ce11e998ec7cd30ad32e2e	1
8	1.00	d311268343ce11e998ec7cd30ad32e2e	2
9	1.00	d311268343ce11e998ec7cd30ad32e2e	3
10	1.00	d311268343ce11e998ec7cd30ad32e2e	4
11	0.01	d311268343ce11e998ec7cd30ad32e2e	5
12	0.01	d311268343ce11e998ec7cd30ad32e2e	6
13	0.01	d311268343ce11e998ec7cd30ad32e2e	7
14	0.01	d311268343ce11e998ec7cd30ad32e2e	8
15	0.01	d311268343ce11e998ec7cd30ad32e2e	9
16	0.01	d311268343ce11e998ec7cd30ad32e2e	10
17	79.90	b169106f695311e998ec7cd30ad32e2e	1
18	53.57	b169106f695311e998ec7cd30ad32e2e	2
19	53.57	b169106f695311e998ec7cd30ad32e2e	3
20	53.57	b169106f695311e998ec7cd30ad32e2e	4
21	53.57	b169106f695311e998ec7cd30ad32e2e	5
22	44.56	b169106f695311e998ec7cd30ad32e2e	6
23	44.56	b169106f695311e998ec7cd30ad32e2e	7
24	44.56	b169106f695311e998ec7cd30ad32e2e	8
25	44.56	b169106f695311e998ec7cd30ad32e2e	9
26	44.56	b169106f695311e998ec7cd30ad32e2e	10
27	165.60	7f12b5164c4411e998ec7cd30ad32e2e	1
28	89.20	7f12b5164c4411e998ec7cd30ad32e2e	2
29	89.20	7f12b5164c4411e998ec7cd30ad32e2e	3
30	89.20	7f12b5164c4411e998ec7cd30ad32e2e	4
31	89.20	7f12b5164c4411e998ec7cd30ad32e2e	5
32	89.20	7f12b5164c4411e998ec7cd30ad32e2e	6
33	56.10	7f12b5164c4411e998ec7cd30ad32e2e	7
34	47.66	7f12b5164c4411e998ec7cd30ad32e2e	8
35	23.70	7f12b5164c4411e998ec7cd30ad32e2e	9
36	18.93	7f12b5164c4411e998ec7cd30ad32e2e	10





1.2.1.2 RANK

返回值在一组值中的排名。当存在相同金额的订单时，ROW_NUMBER()会依次进行排序，每行数据的序号不相同，而Rank()则不一样，出现相同的金额时，他们的排名是一样的。

Demo1. 获取金额rank排名Top10订单

```
select *
from (
    select order_amount,
           ROW_NUMBER() OVER(order by order_amount desc) rn,
           RANK() OVER(order by order_amount desc) rn_rank
    from yp_dwb.dwb_order_detail
)
where rn_rank <=10
;
```

	order_amount	rn	rn_rank
1	198.00	1	1
2	165.60	2	2
3	149.40	3	3
4	138.00	4	4
5	134.52	5	5
6	126.70	6	6
7	126.70	7	6
8	126.70	8	6
9	126.70	9	6
10	126.70	10	6

Demo2. 获取各店铺金额rank排名Top10订单

```
select *
from (
    select order_amount, store_id,
           ROW_NUMBER() OVER(partition by store_id order by order_amount desc) rn,
           RANK() OVER(partition by store_id order by order_amount desc) rn_rank
    from yp_dwb.dwb_order_detail
)
where rn_rank <=10
;
```



	order_amount	store_id	rn	rn_rank
1	0.75	3922381a5b6611e998ec7cd30ad32e2e	1	1
2	0.02	3922381a5b6611e998ec7cd30ad32e2e	2	2
3	0.02	3922381a5b6611e998ec7cd30ad32e2e	3	2
4	0.02	3922381a5b6611e998ec7cd30ad32e2e	4	2
5	0.02	3922381a5b6611e998ec7cd30ad32e2e	5	2
6	0.02	3922381a5b6611e998ec7cd30ad32e2e	6	2
7	7.20	1e95df943a6f11e998ec7cd30ad32e2e	1	1
8	6.00	1e95df943a6f11e998ec7cd30ad32e2e	2	2
9	5.00	1e95df943a6f11e998ec7cd30ad32e2e	3	3
10	5.00	1e95df943a6f11e998ec7cd30ad32e2e	4	3
11	3.50	1e95df943a6f11e998ec7cd30ad32e2e	5	5
12	7.80	7b09b44e5b6d11e998ec7cd30ad32e2e	1	1
13	7.80	7b09b44e5b6d11e998ec7cd30ad32e2e	2	1
14	6.79	7b09b44e5b6d11e998ec7cd30ad32e2e	3	3
15	6.79	7b09b44e5b6d11e998ec7cd30ad32e2e	4	3
16	2.99	7b09b44e5b6d11e998ec7cd30ad32e2e	5	5
17	2.89	7b09b44e5b6d11e998ec7cd30ad32e2e	6	6
18	2.89	7b09b44e5b6d11e998ec7cd30ad32e2e	7	6
19	2.89	7b09b44e5b6d11e998ec7cd30ad32e2e	8	6
20	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	9	9
21	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	10	9
22	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	11	9
23	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	12	9
24	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	13	9
25	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	14	9
26	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	15	9
27	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	16	9
28	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	17	9
29	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	18	9
30	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	19	9
31	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	20	9
32	1.99	7b09b44e5b6d11e998ec7cd30ad32e2e	21	9
33	118.75	b56290c3512611e998ec7cd30ad32e2e	1	1
34	118.75	b56290c3512611e998ec7cd30ad32e2e	2	1
35	118.75	b56290c3512611e998ec7cd30ad32e2e	3	1
36	118.75	b56290c3512611e998ec7cd30ad32e2e	4	1
37	118.75	b56290c3512611e998ec7cd30ad32e2e	5	1
38	118.75	b56290c3512611e998ec7cd30ad32e2e	6	1

1.2.1.3 DENSE_RANK

返回值在一组值中的排名。这与rank()相似，不同之处在于排名的连续性，DENSE_RANK()排名是连续的，RANK()是跳跃的排名，所以一般情况下用的排名函数就是RANK()。

Demo1: 获取金额DENSE_RANK排名Top10订单

```
select *
from (
    select order_amount,
           ROW_NUMBER() OVER(order by order_amount desc) rn,
           RANK() OVER(order by order_amount desc) rn_rank,
           DENSE_RANK() over(order by order_amount desc) rn_DENSE_RANK
    from yp_dwb.dwb_order_detail
)
where rn_DENSE_RANK <=10
;
```





	order_amount	rn	rn_rank	rn_DENSE_RANK
1	198.00	1	1	1
2	165.60	2	2	2
3	149.40	3	3	3
4	138.00	4	4	4
5	134.52	5	5	5
6	126.70	6	6	6
7	126.70	7	6	6
8	126.70	8	6	6
9	126.70	9	6	6
10	126.70	10	6	6
11	126.52	11	11	7
12	126.52	12	11	7
13	123.50	13	13	8
14	123.50	14	13	8
15	123.50	15	13	8
16	123.50	16	13	8
17	123.50	17	13	8
18	123.50	18	13	8
19	123.50	19	13	8
20	123.50	20	13	8
21	122.90	21	21	9
22	122.90	22	21	9
23	122.90	23	21	9
24	122.90	24	21	9
25	122.78	25	25	10
26	122.78	26	25	10
27	122.78	27	25	10
28	122.78	28	25	10
29	122.78	29	25	10
30	122.78	30	25	10

Demo2. 获取各店铺金额DENSE_RANK排名Top10订单

```
select *
from (
    select order_amount, store_id,
        ROW_NUMBER() OVER(partition by store_id order by order_amount desc) rn,
        RANK() OVER(partition by store_id order by order_amount desc) rn_rank,
        DENSE_RANK() over(partition by store_id order by order_amount desc) rn_DENSE_RANK
    from yp_dwb.dwb_order_detail
)
where rn <=10
;
```



	order_amount	store_id	rn	rn_rank	rn_DENSE_RANK
1	113.60	9822e6b4505c11e998ec7cd30ad32e2e	1	1	1
2	59.09	9822e6b4505c11e998ec7cd30ad32e2e	2	2	2
3	59.09	9822e6b4505c11e998ec7cd30ad32e2e	3	2	2
4	59.09	9822e6b4505c11e998ec7cd30ad32e2e	4	2	2
5	59.09	9822e6b4505c11e998ec7cd30ad32e2e	5	2	2
6	59.09	9822e6b4505c11e998ec7cd30ad32e2e	6	2	2
7	59.09	9822e6b4505c11e998ec7cd30ad32e2e	7	2	2
8	59.09	9822e6b4505c11e998ec7cd30ad32e2e	8	2	2
9	55.00	9822e6b4505c11e998ec7cd30ad32e2e	9	9	3
10	50.00	9822e6b4505c11e998ec7cd30ad32e2e	10	10	4
11	46.91	9822e6b4505c11e998ec7cd30ad32e2e	11	11	5
12	46.56	9822e6b4505c11e998ec7cd30ad32e2e	12	12	6
13	46.56	9822e6b4505c11e998ec7cd30ad32e2e	13	12	6
14	43.99	9822e6b4505c11e998ec7cd30ad32e2e	14	14	7
15	43.99	9822e6b4505c11e998ec7cd30ad32e2e	15	14	7
16	43.99	9822e6b4505c11e998ec7cd30ad32e2e	16	14	7
17	43.99	9822e6b4505c11e998ec7cd30ad32e2e	17	14	7
18	43.99	9822e6b4505c11e998ec7cd30ad32e2e	18	14	7
19	43.99	9822e6b4505c11e998ec7cd30ad32e2e	19	14	7
20	35.13	9822e6b4505c11e998ec7cd30ad32e2e	20	20	8
21	35.13	9822e6b4505c11e998ec7cd30ad32e2e	21	20	8
22	35.13	9822e6b4505c11e998ec7cd30ad32e2e	22	20	8
23	35.13	9822e6b4505c11e998ec7cd30ad32e2e	23	20	8
24	29.70	9822e6b4505c11e998ec7cd30ad32e2e	24	24	9
25	29.70	9822e6b4505c11e998ec7cd30ad32e2e	25	24	9
26	29.70	9822e6b4505c11e998ec7cd30ad32e2e	26	24	9
27	29.70	9822e6b4505c11e998ec7cd30ad32e2e	27	24	9
28	19.80	9822e6b4505c11e998ec7cd30ad32e2e	28	28	10

1.2.1.4 NTILE

NTILE()函数它把有序的数据集合 平均分配 到 指定的数量 (num) 个组中, 如果不能平均分配, 则优先分配较小编号的组, 并且各个组中能放的行数最多相差1。各个组有编号, 编号从1开始, 就像我们说的'分区'一样, 分为几个区, 一个区会有多少个。

每个组行数最多相差1的含义:

- 1、每组的记录数不能大于它上一组的记录数, 即编号小的桶放的记录数不能小于编号大的桶。也就是说, 第1组中的记录数只能大于等于第2组及以后各组中的记录数。
- 2、所有组中的记录数要么都相同, 要么从某一个记录较少的组 (命名为X) 开始后面所有组的记录数都与该组 (X组) 的记录数相同。也就是说, 如果有个组, 前三组的记录数都是9, 而第四组的记录数是8, 那么第五组和第六组的记录数也必须是8。

Demo1. 获取金额NTILE500的Top10订单

```
select *
from (
    select order_amount,
           ROW_NUMBER() OVER(order by order_amount desc) rn,
           RANK() OVER(order by order_amount desc) rn_rank,
           DENSE_RANK() over(order by order amount desc) rn_DENSE_RANK,
           NTILE(500) over(order by order_amount desc) rn_NTILE
```





```
from yp_dwb.dwb_order_detail
)
where rn_NTILE <=10
;
```

意思：根据order_amount倒序排序后，尽量地平均分5(500)组。

例如：如果有4个值，则会把这4个值分成4组，4,3,2,1，共4堆；

如果是5个值，则分成5,4,3,2,1，共5堆；

如果是6个值，则分成 (6,5) ,4,3,2,1,共5堆；

如果是7个值，则分成 (7,6) , (5,4) ,3,2,1,共5堆。

	order_amount	rn	rn_rank	rn_DENSE_RANK	rn_NTILE
1	198.00	1	1	1	1
2	165.60	2	2	2	1
3	149.40	3	3	3	1
4	138.00	4	4	4	1
5	134.52	5	5	5	1
6	126.70	6	6	6	1
7	126.70	7	6	6	1
8	126.70	8	6	6	1
9	126.70	9	6	6	1
10	126.70	10	6	6	1
11	126.52	11	11	7	2
12	126.52	12	11	7	2
13	123.50	13	13	8	2
14	123.50	14	13	8	2
15	123.50	15	13	8	2
16	123.50	16	13	8	2
17	123.50	17	13	8	2
18	123.50	18	13	8	2
19	123.50	19	13	8	2
20	123.50	20	13	8	2
21	122.90	21	21	9	3
22	122.90	22	21	9	3
23	122.90	23	21	9	3
24	122.90	24	21	9	3
25	122.78	25	25	10	3
26	122.78	26	25	10	3
27	122.78	27	25	10	3
28	122.78	28	25	10	3
29	122.78	29	25	10	3
30	122.78	30	25	10	3
31	121.25	31	31	11	4
32	121.25	32	31	11	4
33	121.25	33	31	11	4
34	121.25	34	31	11	4
35	121.25	35	31	11	4
36	121.25	36	31	11	4
37	121.25	37	31	11	4
38	119.92	38	38	12	4





Demo2. 获取各店铺金额NTILE500的Top10订单

```
select *
from (
    select order_amount, store_id,
        ROW_NUMBER() OVER(partition by store_id order by order_amount desc) rn,
        RANK() OVER(partition by store_id order by order_amount desc) rn_rank,
        DENSE_RANK() over(partition by store_id order by order_amount desc) rn_DENSE_RANK,
        NTILE(500) over(partition by store_id order by order_amount desc) rn_NTILE
    from yp_dwb.dwb_order_detail
)
where rn_NTILE <=10
;
```

意思：根据 store_id 分组(groupby)，每个店铺内再按照order_amount倒序排序分组(NTILE)，店铺内部尽量地平均分为5(500)组。

例如：如果有4个值，则会把这4个值分成4组，4,3,2,1，共4堆；

如果是5个值，则分成5,4,3,2,1，共5堆；

如果是6个值，则分成 (6,5) ,4,3,2,1,共5堆；

如果是7个值，则分成 (7,6) , (5,4) ,3,2,1,共5堆。

此处如果将NTILE参数改为1，同一个store下的数据都会分到1这个分区下。



	order_amount	store_id	rn	rn_rank	rn_DENSE_RANK	rn_NTILE
1	41.78	1292c04f3af811e998ec7cd30ad32e2e	1	1	1	1
2	41.78	1292c04f3af811e998ec7cd30ad32e2e	2	1	1	1
3	21.90	1292c04f3af811e998ec7cd30ad32e2e	3	3	2	1
4	20.00	1292c04f3af811e998ec7cd30ad32e2e	4	4	3	1
5	16.90	1292c04f3af811e998ec7cd30ad32e2e	5	5	4	1
6	16.90	1292c04f3af811e998ec7cd30ad32e2e	6	5	4	1
7	15.00	1292c04f3af811e998ec7cd30ad32e2e	7	7	5	1
8	11.00	1292c04f3af811e998ec7cd30ad32e2e	8	8	6	1
9	8.00	1292c04f3af811e998ec7cd30ad32e2e	9	9	7	1
10	6.00	1292c04f3af811e998ec7cd30ad32e2e	10	10	8	1
11	6.00	1292c04f3af811e998ec7cd30ad32e2e	11	10	8	1
12	6.00	1292c04f3af811e998ec7cd30ad32e2e	12	10	8	1
13	5.90	1292c04f3af811e998ec7cd30ad32e2e	13	13	9	1
14	5.90	1292c04f3af811e998ec7cd30ad32e2e	14	13	9	1
15	5.60	1292c04f3af811e998ec7cd30ad32e2e	15	15	10	1
16	5.00	1292c04f3af811e998ec7cd30ad32e2e	16	16	11	1
17	5.00	1292c04f3af811e998ec7cd30ad32e2e	17	16	11	1
18	5.00	1292c04f3af811e998ec7cd30ad32e2e	18	16	11	1
19	5.00	1292c04f3af811e998ec7cd30ad32e2e	19	16	11	1
20	4.59	1292c04f3af811e998ec7cd30ad32e2e	20	20	12	1
21	4.59	1292c04f3af811e998ec7cd30ad32e2e	21	20	12	1
22	3.50	1292c04f3af811e998ec7cd30ad32e2e	22	22	13	1
23	3.50	1292c04f3af811e998ec7cd30ad32e2e	23	22	13	1
24	3.50	1292c04f3af811e998ec7cd30ad32e2e	24	22	13	1
25	3.50	1292c04f3af811e998ec7cd30ad32e2e	25	22	13	1
26	0.01	cdef4e0b4ac211e998ec7cd30ad32e2e	1	1	1	1
27	0.01	cdef4e0b4ac211e998ec7cd30ad32e2e	2	1	1	1
28	58.49	52728bdb3c9b11e998ec7cd30ad32e2e	1	1	1	1
29	49.00	52728bdb3c9b11e998ec7cd30ad32e2e	2	2	2	1
30	18.50	52728bdb3c9b11e998ec7cd30ad32e2e	3	3	3	1
31	12.99	52728bdb3c9b11e998ec7cd30ad32e2e	4	4	4	1
32	12.99	52728bdb3c9b11e998ec7cd30ad32e2e	5	4	4	1
33	2.50	52728bdb3c9b11e998ec7cd30ad32e2e	6	6	5	1
34	1.01	9174cce6407911e998ec7cd30ad32e2e	1	1	1	1
35	60.19	17eb26e8512b11e998ec7cd30ad32e2e	1	1	1	1
36	60.19	17eb26e8512b11e998ec7cd30ad32e2e	2	1	1	1
37	60.19	17eb26e8512b11e998ec7cd30ad32e2e	3	1	1	1
38	28.00	17eb26e8512b11e998ec7cd30ad32e2e	4	4	2	1

1.2.2 按城市分组统计

1.2.2.1 表关系

按照城市、所在省份和日期分组统计。

指标和维度字段分别来源于：订单明细宽表、店铺明细宽表、商品明细宽表。

```
select *
from yp_dwb.dwb_order_detail o
left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
where o.is_valid=1
```





1.2.2.2 配送费字段选择

```
select o.dispatcher_money, o.delivery_fee
from yp_dwb.dwb_order_detail_test o
where o.delivery_fee != 0;
```

	123 dispatcher_money	123 delivery_fee
37	0	3
38	0	1.1
39	0	0.32
40	0	0.32
41	0	1.46
42	0	1.46
43	0	1.46
44	0	1.46
45	0	1.83
46	0	3
47	0	4.91
48	0	0.31

可以看出，来自fact_order_delivery_item表的delivery_fee字段数据较全，实现时要与需求方确认。

1.2.2.3 SQL

sum内判断，不计算时可返回0；count内判断，不计算时可返回null；

可以利用row_number分组排序的特性，将订单去重后统计订单量和订单金额。

在复杂计算的sql中，可以利用多个row_number的分组排序来实现不同的分组去重效果。

```
-- 分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
-- 写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
-- 分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;
set hive.optimize.bucketmapjoin = true;
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;
```





```
with order_base as
(
    select
-- 维度
        SUBSTRING(o.create_date,1,10) as dt,
        province_id,
        province_name,
        s.city_id,
        s.city_name,
-- 订单量指标
        o.order_id,
-- 金额指标
        o.order_amount,
        o.plat_fee,
        o.delivery_fee,
-- 判断条件
        o.order_from,
        o.evaluation_id,
        o.geval_scores,
        o.delievery_id,
        o.refund_id,
        row_number() over(partition by o.order_id rn
        from yp_dwb.dwb_order_detail o
        left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
        left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
        where o.is_pay=1
-- and o.order_id='dd190404391438bed2'
    )
insert into yp_dws.dws_sale_daycount partition (dt)
select
-- =====维度=====
    province_id,
    province_name,
    city_id,
    city_name,
    null as trade_area_id,
    null as trade_area_name,
    null as store_id,
    null as store_name,
    null as brand_id,
    null as brand_name,
```





```

null as max_class_id,
null as max_class_name,
null as mid_class_id,
null as mid_class_name,
null as min_class_id,
null as min_class_name,
'city' as group_type
-- =====金额=====
-- 销售收入
,sum(coalesce(order_amount, 0)) sale_amt
-- 平台收入
,sum(coalesce(plat_fee,0)) plat_amt
-- 配送成交额
,sum(coalesce(delivery_fee,0)) deliver_sale_amt
-- 小程序成交额
,sum(if(order_from='miniapp', coalesce(order_amount, 0), 0)) mini_app_sale_amt
-- 安卓APP成交额
,sum(if(order_from='android', coalesce(order_amount, 0), 0)) android_sale_amt
-- 苹果APP成交额
,sum(if(order_from='ios', coalesce(order_amount, 0), 0)) ios_sale_amt
-- PC商城成交额
,sum(if(order_from='pcweb', coalesce(order_amount, 0), 0)) pcweb_sale_amt
-- =====订单量=====
-- 单量
,count(order_id) order_cnt
-- 参评单量
,count(if(evaluation_id is not null, order_id, null)) eva_order_cnt
-- 差评单量
,count(if(evaluation_id is not null and geval_scores<=6, order_id, null)) bad_eva_order_cnt
-- 配送单量
,count(if(delievery_id is not null, order_id, null)) deliver_order_cnt
-- 退款单量
,count(if(refund_id is not null, order_id, null)) refund_order_cnt
-- 小程序单量
,count(if(order_from='miniapp', order_id, null)) miniapp_order_cnt
-- 安卓APP订单量
,count(if(order_from='android', order_id, null)) android_order_cnt
-- 苹果APP订单量
,count(if(order_from='ios', order_id, null)) ios_order_cnt
```



```
-- PC商城单量
, count(if(order_from='pcweb', order_id, null)) pcweb_order_cnt
, dt
from order_base b
where b.rn=1
group by dt, city_id, city_name, province_id, province_name
;
```

1.2.3 数据倾斜

如果数据量很大或者出现了数据倾斜比较严重的情况，如何来优化呢？

1.2.3.1 map join、Bucket-Map join、SMB join

详见意向客户主题看板。

1.2.3.2 表连接数据倾斜(Join skew in)

1.2.3.2.1 运行时优化

set hive.optimize.skewjoin=true;

默认关闭。

如果大表和大表进行join操作，则可采用skewjoin(倾斜关联)来开启对倾斜数据的优化。

skewjoin原理：

1. 对于skewjoin.key，在执行job时，将它们存入临时的HDFS目录，其它数据正常执行
2. 对倾斜数据开启map join操作（多个map并行处理），对非倾斜值采取普通join操作
3. 将倾斜数据集和非倾斜数据集进行合并Union操作。

开启skewin以后，究竟多大的数据才会被认为是倾斜了的数据呢？

set hive.skewjoin.key=100000;

默认值100000。



如果join的key对应的记录条数超过这个值，就认为这个key产生了数据倾斜，则会对其进行分拆优化。

1.2.3.2.2 编译时优化

上面的配置项其实应该理解为hive.optimize.skewjoin.runtime，也就是sql运行时来对偏斜信息进行优化；除此之外还有另外一个配置：

```
set hive.optimize.skewjoin.compiletime=true;
```

默认关闭。

此参数的用处和上面的hive.optimize.skewjoin一致，但在编译sql时就已经将执行计划优化完毕。但要注意的是，只有在表的元数据中存储的有数据倾斜信息时，才能生效。因此建议runtime和compiletime都设置为true。

可以通过建表语句来指定数据倾斜元数据：

```
CREATE TABLE list_bucket_single (key STRING, value STRING)
-- 倾斜的字段和需要拆分的key值
SKEWED BY (key) ON (1,5,6)
-- 为倾斜值创建子目录单独存放
[STORED AS DIRECTORIES];
```

1.2.3.2.3 Union优化

应用了表连接倾斜优化以后，会在执行计划中插入一个新的union操作，此时建议开启对union的优化配置：

```
set hive.optimize.union.remove=true;
```

默认关闭。

此项配置减少对Union all子查询中间结果的二次读写，可以避免union输出的额外扫描过程，当我们开启了skewjoin时尤其有用，建议同时开启。

```
set hive.optimize.skewjoin=true;
set hive.optimize.skewjoin.compiletime=true;
set hive.optimize.union.remove=true;
```





1.2.3.3 分组统计数据倾斜(Groupby skew in)

1.2.3.3.1 Map阶段聚合

```
set hive.map.aggr=true;
```

开启map端combiner。此配置可以在group by语句中提高HiveQL聚合的执行性能。这个设置可以将顶层的聚合操作放在Map阶段执行，从而减轻数据传输和Reduce阶段的执行时间，提升总体性能。默认开启，无需显示声明。

启用 Map 端聚合
hive.map.aggr

☒ HiveServer2 Default Group

1.2.3.3.2 MRJob随机数打散

```
set hive.groupby.skewindata=true;
```

默认关闭。

这个配置项是用于决定group by操作是否支持倾斜数据的负载均衡处理。当数据出现倾斜时，如果该变量设置为true，那么Hive会自动进行负载均衡。

当选项设定为 true，生成的查询计划会有两个 MR Job。

第一个MR Job中，Map 的输出结果集合会随机分布到Reduce中，每个Reduce做部分聚合操作，并输出结果，这样处理的结果是相同的Group By Key有可能被分发到不同的Reduce中，从而达到负载均衡的目的；

第二个MR Job再根据预处理的数据结果按照Group By Key分布到Reduce中（这个过程可以保证相同的Group By Key被分布到同一个Reduce中），最后完成最终的聚合操作。

注意：在多个列上进行的去重操作与hive环境变量hive.groupby.skewindata存在冲突。

当hive.groupby.skewindata=true时，hive不支持多列上的去重操作，并报错：

Error in semantic analysis: DISTINCT on different columns not supported with skew in data.

比如：



- (1) SELECT count(DISTINCT uid) FROM log
- (2) SELECT ip, count(DISTINCT uid) FROM log GROUP BY ip
- (3) SELECT ip, count(DISTINCT uid, uname) FROM log GROUP BY ip
- (4) SELECT ip, count(DISTINCT uid), count(DISTINCT uname) FROM log GROUP BY ip

1、2、3能够正常执行，但是4会报错。

1.2.4 按商圈分组统计

按照**商圈**、**日期**分组。

```
--分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
--写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
--分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;
set hive.optimize.bucketmapjoin = true;
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;
--并行执行
set hive.exec.parallel=true;
set hive.exec.parallel.thread.number=8;
--join数据倾斜
set hive.optimize.skewjoin=true;
-- set hive.skewjoin.key=100000;
set hive.optimize.skewjoin.compiletime=true;
set hive.optimize.union.remove=true;
--group倾斜
set hive.groupby.skewindata=true;

with order_base as
(
    select
-- 维度
        SUBSTRING(o.create_date,1,10) as dt,
        province_id,
        province_name,
```





```
s.city_id,  
s.city_name,  
trade_area_id, trade_area_name,  
-- 订单量指标  
o.order_id,  
-- 金额指标  
o.order_amount,  
o.plat_fee,  
o.delivery_fee,  
-- 判断条件  
o.order_from,  
o.evaluation_id,  
o.geval_scores,  
o.delievery_id,  
o.refund_id,  
row_number() over(partition by o.order_id) rn  
from yp_dwb.dwb_order_detail o  
left join yp_dwb.dwb_shop_detail s on o.store_id=s.id  
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id  
where o.is_pay=1  
-- and o.order_id='dd190404391438bed2'  
)  
insert into yp_dws.dws_sale_daycount partition (dt)  
select  
-- =====维度=====  
province_id,  
province_name,  
city_id,  
city_name,  
trade_area_id,  
trade_area_name,  
null as store_id,  
null as store_name,  
null as brand_id,  
null as brand_name,  
null as max_class_id,  
null as max_class_name,  
null as mid_class_id,  
null as mid_class_name,  
null as min_class_id,  
null as min_class_name,
```





```
'city' as group_type
-- =====金额=====
-- 销售收入
, sum(coalesce(order_amount, 0)) sale_amt
-- 平台收入
, sum(coalesce(plat_fee, 0)) plat_amt
-- 配送成交额
, sum(coalesce(delivery_fee, 0)) deliver_sale_amt
-- 小程序成交额
, sum(if(order_from='miniapp', coalesce(order_amount, 0), 0)) mini_app_sale_amt
-- 安卓APP成交额
, sum(if(order_from='android', coalesce(order_amount, 0), 0)) android_sale_amt
-- 苹果APP成交额
, sum(if(order_from='ios', coalesce(order_amount, 0), 0)) ios_sale_amt
-- PC商城成交额
, sum(if(order_from='pcweb', coalesce(order_amount, 0), 0)) pcweb_sale_amt
-- =====订单量=====
-- 单量
, count(order_id) order_cnt
-- 参评单量
, count(if(evaluation_id is not null, order_id, null)) eva_order_cnt
-- 差评单量
, count(if(evaluation_id is not null and geval_scores<=6, order_id, null)) bad_eva_order_cnt
-- 配送单量
, count(if(delievery_id is not null, order_id, null)) deliver_order_cnt
-- 退款单量
, count(if(refund_id is not null, order_id, null)) refund_order_cnt
-- 小程序单量
, count(if(order_from='miniapp', order_id, null)) miniapp_order_cnt
-- 安卓APP订单量
, count(if(order_from='android', order_id, null)) android_order_cnt
-- 苹果APP订单量
, count(if(order_from='ios', order_id, null)) ios_order_cnt
-- PC商城单量
, count(if(order_from='pcweb', order_id, null)) pcweb_order_cnt
, dt
from order_base b
where b.rn=1
group by dt, trade_area_id, trade_area_name, city_id, city_name, province_id, province_name
```



1.2.5 按照品牌分组统计

品牌与订单的关系，与商圈和城市的1对多不同，是多对多的关系，如图：

```
CREATE TABLE yp_dws.dws_sale_daycount(
  city_id string COMMENT '城市id',
  city_name string COMMENT '城市名称',
  trade_area_id string COMMENT '商圈id',
  trade_area_name string COMMENT '商圈名称',
  store_id string COMMENT '店铺id',
  store_name string COMMENT '店铺名称',
  brand_id string COMMENT '品牌id',
  brand_name string COMMENT '品牌名称',
  max_class_id string COMMENT '商品大类id',
  max_class_name string COMMENT '大类名称',
  mid_class_id string COMMENT '中类id',
  mid_class_name string COMMENT '中类名称',
  min_class_id string COMMENT '小类id',
  min_class_name string COMMENT '小类名称',
  group_type string COMMENT '分组类型: store, trade_area, city, brand, min_class, mid_c',
  -- =====日统计=====
  -- 销售收入
  sale_amt DECIMAL(38,2) COMMENT '销售收入',
  -- 平台收入
  plat_amt DECIMAL(38,2) COMMENT '平台收入',
  -- 配送成交额
  deliver_sale_amt DECIMAL(38,2) COMMENT '配送成交额',
  -- 小程序成交额
  mini_app_sale_amt DECIMAL(38,2) COMMENT '小程序成交额',
  -- 安卓APP成交额
  android_sale_amt DECIMAL(38,2) COMMENT '安卓APP成交额',
  -- 苹果APP成交额
  ios_sale_amt DECIMAL(38,2) COMMENT '苹果APP成交额',
  -- PC商城成交额
  pcweb_sale_amt DECIMAL(38,2) COMMENT 'PC商城成交额',
  -- 成交单量
  order_cnt BIGINT COMMENT '成交单量',
  -- 参评单量
  eva_order_cnt BIGINT COMMENT '参评单量comment=>cmt',
  -- 差评单量
```

一个店铺下有多个订单；
一个订单只能属于一个店铺 (1:n)

一个品牌有多个订单；
一个订单有多个品牌的商品 (n:n)

一个小分类有多个订单；
一个订单有多个分类商品 (n:n)

订单

因此，按照品牌计算销售额时，不能直接累加整个订单的金额，而是要计算订单中品牌商品的金额，因此要按照goods_id分组。注意此时的订单量统计，需要按照订单id去重。

```
-- 分区
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;
--hive压缩
set hive.exec.compress.intermediate=true;
set hive.exec.compress.output=true;
```





```
-- 写入时压缩生效
set hive.exec.orc.compression.strategy=COMPRESSION;
-- 分桶
set hive.enforce.bucketing=true;
set hive.enforce.sorting=true;
set hive.optimize.bucketmapjoin = true;
set hive.auto.convert.sortmerge.join=true;
set hive.auto.convert.sortmerge.join.noconditionaltask=true;
-- 并行执行
set hive.exec.parallel=true;
set hive.exec.parallel.thread.number=8;
-- join 数据倾斜
set hive.optimize.skewjoin=true;
-- set hive.skewjoin.key=100000;
set hive.optimize.skewjoin.compiletime=true;
set hive.optimize.union.remove=true;
-- group 倾斜
set hive.groupby.skewindata=true;

with order_base as
(
    select
        -- 维度
        SUBSTRING(o.create_date,1,10) as dt,
        brand_id, brand_name,
        -- 订单量指标
        o.order_id,
        -- 金额指标
        o.order_amount,
        o.plat_fee,
        o.delivery_fee,
        -- 判断条件
        o.order_from,
        o.evaluation_id,
        o.geval_scores,
        o.delievery_id,
        o.refund_id,
        row_number() over(partition by g.brand_id, o.order_id, o.goods_id) brand_rn
    from yp_dwb.dwb_order_detail o
        left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
        left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
    where o.is_pay=1
    -- and o.order_id='dd190404391438bed2'
)
-- insert into yp_dws.dws_sale_daycount partition (dt)
select
```





```
-- =====维度=====
null as province_id,
null as province_name,
null as city_id,
null as city_name,
null as trade_area_id,
null as trade_area_name,
null as store_id,
null as store_name,
brand_id,
brand_name,
null as max_class_id,
null as max_class_name,
null as mid_class_id,
null as mid_class_name,
null as min_class_id,
null as min_class_name,
'city' as group_type
-- =====金额=====
-- 销售收入
,sum(coalesce(order_amount, 0)) sale_amt
-- 平台收入
,sum(coalesce(plat_fee,0)) plat_amt
-- 配送成交额
,sum(coalesce(delivery_fee,0)) deliver_sale_amt
-- 小程序成交额
,sum(if(order_from='miniapp', coalesce(order_amount, 0), 0)) mini_app_sale_amt
-- 安卓APP成交额
,sum(if(order_from='android', coalesce(order_amount, 0), 0)) android_sale_amt
-- 苹果APP成交额
,sum(if(order_from='ios', coalesce(order_amount, 0), 0)) ios_sale_amt
-- PC商城成交额
,sum(if(order_from='pcweb', coalesce(order_amount, 0), 0)) pcweb_sale_amt
-- =====订单量=====
-- 单量
,count(distinct order_id) order_cnt
-- 参评单量
,count(distinct if(evaluation_id is not null, order_id, null)) eva_order_cnt
-- 差评单量
,count(distinct if(evaluation_id is not null and geval_scores<=6, order_id, null))
```





```
bad_eva_order_cnt
-- 配送单量
,count(distinct if(delievery_id is not null, order_id, null)) deliver_order_cnt
-- 退款单量
,count(distinct if(refund_id is not null, order_id, null)) refund_order_cnt
-- 小程序单量
,count(distinct if(order_from='miniapp', order_id, null)) miniapp_order_cnt
-- 安卓APP订单量
,count(distinct if(order_from='android', order_id, null)) android_order_cnt
-- 苹果APP订单量
,count(distinct if(order_from='ios', order_id, null)) ios_order_cnt
-- PC商城单量
,count(distinct if(order_from='pcweb', order_id, null)) pcweb_order_cnt
,dt
from order_base b
where b.brand_rn=1
group by dt, brand_id, brand_name
order by order_cnt desc, brand_id
;
```

Error while compiling statement: FAILED: SemanticException [Error 10022]: DISTINCT on different columns not supported with skew in data

DISTINCT on different columns not supported with skew in data。还记得这个错误提示吗？

1.2.6 其他维度分组统计

略，店铺维度和商圈维度相似，大类中类小类和品牌维度相似。

1.2.7 Hive其他优化

1.2.7.1 关联优化器

在Hive的一些**复杂关联查询**中，可能同时还包含有**group by**等能够触发shuffle的操作，有些时候shuffle操作是可以共享的，通过关联优化器选项，可以尽量**减少复杂查询中的shuffle**，从而提升



性能。

```
set hive.optimize.correlation=true;
```

1.2.7.2 Hive并行执行

Hive会将一个查询转化为一个或多个阶段，包括：MapReduce阶段、抽样阶段、合并阶段、limit阶段等。默认情况下，一次只执行一个阶段。不过，如果某些阶段不是互相依赖，是可以并行执行的。

set hive.exec.parallel=true,可以开启并发执行，默认为false。

set hive.exec.parallel.thread.number=16; //同一个sql允许的最大并行度，默认为8。

```
set hive.exec.parallel=true;  
set hive.exec.parallel.thread.number=16;
```

1.2.7.3 Hive小文件合并

此部分设置，要根据硬件内存来进行调整，**个人电脑配置较低**，不建议修改。

hive.merge.mapfiles

是否开启合并Map端小文件，在**Map-only**的任务结束时合并小文件，true是打开。

hive.merge.mapredfiles

是否开启合并Reduce端小文件，在**map-reduce**作业结束时合并小文件。true是打开。

hive.merge.size.per.task

合并后MR输出文件的大小，默认为256M。

hive.merge.smallfiles.avgsize

当输出文件的平均大小**小于此设置值时**，启动一个独立的map-reduce任务进行**文件merge**，默认值为16M。



启用小文件合并 - Map-Only 作业

hive.merge.mapfiles

☒ HiveServer2 Default Group

启用小文件合并 - Map-Reduce 作业

hive.merge.mapredfiles

☒ HiveServer2 Default Group

启用小文件合并 - Spark 作业

hive.merge.sparkfiles

☒ HiveServer2 Default Group

小文件平均大小合并阈值

hive.merge.smallfiles.avgsize

HiveServer2 Default Group

512

兆字节

合并后所需的文件大小

hive.merge.size.per.task

HiveServer2 Default Group

512

兆字节

1.2.7.4 矢量化查询

hive的默认查询执行引擎一次处理一行，而矢量化查询执行是一种hive特性，目的是按照每批1024行读取数据，并且一次性对整个记录整合（而不是对单条记录）应用操作，注意：要使用矢量化查询执行，就必须以ORC格式存储数据。

```
set hive.vectorized.execution.enabled=true;
set hive.vectorized.execution.reduce.enabled = true;
```

1.2.7.5 读取零拷贝

ORC可以使用新的HDFS缓存API和ZeroCopy读取器来避免在扫描文件时将额外的数据复制到内存中。

```
set hive.exec.orc.zerocopy=true;
```





1.3 Presto

详见《Home\讲义\第5章 DWS和Presto\4.Presto.docx》。

1.4 Presto实现

由于DWB层实现了宽表，通过把相关字段放在同一张表中，可以大大减轻DWS数据计算时的消耗，提高计算效率问题。在内存资源足够的情况下，推荐使用Presto进行计算。

注：Presto同样支持ORC file的bloom filter index与row group index。

1.4.1 实现

1.4.1.1 多GROUPBY合并

1.4.1.1.1 GROUPING SETS

Grouping sets允许用户指定要分组的多个列列表。将不属于分组列的给定子列表的列设置为NULL。

比如，需求分别按照店铺分组、订单组分组、店铺和订单组分组统计订单销售额，并获取三者的结果集（插入到宽表）。可以通过union all多个group by来实现：

```
select store_id,  
       null,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by store_id  
  
union all  
  
select null,  
       group_id,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by group_id  
  
union all
```





```
select store_id,  
       group_id,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by store_id, group_id;
```

这样的SQL会比较长，当宽表需求的分组维度比较多时，SQL会越来越长。此时可以通过 grouping sets来实现同样的运算，同时减少重复的SQL语句：

```
select store_id,  
       group_id,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by  
       grouping sets (store_id, group_id, (store_id, group_id));
```

同时，使用grouping sets、cube、rollup这些复杂group语法的SQL只会读取基础数据源一次，而使用union all拼接的SQL会读取基础数据源三次（或更多）。因此当数据源正在频繁变化时，使用union all拼接的方式可能会产生一些奇怪的结果。

	store_id	_col1	_col2
1	5d3b50ab601c11e998ec7cd30ad32e2e	[NULL]	23,899.79
2	7b09b44e5b6d11e998ec7cd30ad32e2e	[NULL]	72.69
3	9822e6b4505c11e998ec7cd30ad32e2e	[NULL]	2,460.92
4	4393d58a504b11e998ec7cd30ad32e2e	[NULL]	3,127.54
5	8aa68e0151e811e998ec7cd30ad32e2e	[NULL]	5,424.95
6	3922381a5b6611e998ec7cd30ad32e2e	[NULL]	0.85
7	1e95df943a6f11e998ec7cd30ad32e2e	[NULL]	26.7
8	b56290c3512611e998ec7cd30ad32e2e	[NULL]	2,597.5
9	e547ecda506811e998ec7cd30ad32e2e	[NULL]	4,566.3
10	[NULL]	dd20190401185702249478b91	55
11	[NULL]	dd20190403125208764883e0f	1
12	[NULL]	dd20190403103300886555d78	1
13	[NULL]	dd201904031621488305039e0	1
14	[NULL]	dd2019040411460128342eda3	1
15	[NULL]	dd201904040855247260923f3	34.2

1.4.1.1.2 CUBE

CUBE操作会生成所提供column所有可能的grouping sets结果。





比如：

```
select store_id,  
       group_id,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by  
       cube (store_id,group_id);
```

等同于：

```
select store_id,  
       group_id,  
       sum(coalesce(order_amount,0))  
from yp_dwb.dwb_order_detail  
group by  
       grouping sets ((store_id,group_id),store_id, group_id, ());
```

	store_id	group_id	_col2
278	[NULL]	dd2019041815315204906836e	0.01
279	[NULL]	dd20190420142334187828ca2	3.8
280	8aa68e0151e81	dd2019041216303961985de52	7.6
281	5d3b50ab601c1	dd20190420141844830768ca2	2.98
282	76e80001cdf01	[NULL]	0.05
283	[NULL]	dd2019042109330420050961f	43.8
284	[NULL]	dd20190423082503921054823	4.48
285	[NULL]	dd20190423185206760596454	6.02
286	[NULL]	dd20190517123945252066a95	48
287	[NULL]	dd20190517172205009422d15	9.8
288	[NULL]	dd2019040309034892079c748	1
289	[NULL]	dd20190403145756825319e58	1
290	5d3b50ab601c1	dd20190423203031008781daf	0.01
291	5d3b50ab601c1	dd20190428123236361429758	0.01
292	e547ecda50681	dd20190403091042889055a9d	1

1.4.1.1.3 ROLLUP

rollup操作比较特殊，会逐步进行末位淘汰，直接看例子：

```
select store_id,
```



```
group_id,
sum(coalesce(order_amount,0))
from yp_dwb.dwb_order_detail
group by
rollup (store_id,group_id);
```

等同于：

```
select store_id,
group_id,
sum(coalesce(order_amount,0))
from yp_dwb.dwb_order_detail
group by
grouping sets ((store_id,group_id), (store_id), ());
```

即，rollup((a),(b),(c))等价于grouping sets((a,b,c),(a,b),(a),())。

	store_id	group_id	_col2
1	e547ecda506811e998ec7cd30ad32e2e	dd2019052310434623689442b	31.76
2	17eb26e8512b11e998ec7cd30ad32e2e	dd2019053011445729817836e	2.98
3	d5da2741552611e998ec7cd30ad32e2e	[NULL]	675.68
4	d5da2741552611e998ec7cd30ad32e2e	dd20190421073050188441e27	5
5	d5da2741552611e998ec7cd30ad32e2e	dd2019042715515967333d295	24
6	d5da2741552611e998ec7cd30ad32e2e	dd201904030956341642284aa	1
7	4393d58a504b11e998ec7cd30ad32e2e	dd2019040315202638977ccd2	1
8	e547ecda506811e998ec7cd30ad32e2e	dd20190403125951947291e85	1
9	25d50282552a11e998ec7cd30ad32e2e	dd20190404132017357822a4e	1
10	d5da2741552611e998ec7cd30ad32e2e	dd20190421193900123992dd9	0.01
11	e547ecda506811e998ec7cd30ad32e2e	dd20190519122353716126a7d	2.4
12	4393d58a504b11e998ec7cd30ad32e2e	dd2019042821252238821edb0	0.01
13	5d3b50ab601c11e998ec7cd30ad32e2e	dd2019052119481064016885d	49.5
14	b169106f695311e998ec7cd30ad32e2e	dd20190513194142639775aaf	8.97
15	4393d58a504b11e998ec7cd30ad32e2e	dd2019040309331658270f0dc	1

1.4.1.1.4 GROUPING函数

grouping 返回一个转换为十进制数字的二进制数值集合，指示分组中存在哪些列。它必须与 GROUPING SETS，ROLLUP，CUBE或GROUP BY结合使用，并且其参数必须与相应的GROUPING SETS，ROLLUP，CUBE或GROUP BY子句中引用的列完全匹配。

对于给定的分组，如果分组中包含相应的列，则将位设置为0，否则将其设置为1。

Demo：

```
select group_id,
store_id,
```



```
sum(coalesce(goods_amount,0)),
grouping(group_id) as ging,
grouping(store_id) as sing,
grouping(group_id, store_id) as g_s_ing
from yp_dwb.dwb_order_detail
group by
grouping sets (group_id, store_id, (group_id, store_id));
```

group_id	store_id	col2	ging	sing	g_s_ing
dd201904121845399538885d	<null>	3.20	0	1	1
dd2019041214344197066cd3c	<null>	29.98	0	1	1
dd2019041210183809402a062	<null>	2.99	0	1	1
dd20190413173746480050509	<null>	1.50	0	1	1
dd20190414202417163410af6	<null>	9.90	0	1	1
dd2019041620444230720f761	<null>	12.00	0	1	1
dd2019041609595266194760d	<null>	7.90	0	1	1
dd20190417001048186999050	<null>	7.90	0	1	1
dd2019041816123330295e543	<null>	1.99	0	1	1
<null>	7b09b44e5b6d11e998ec7cd30ad32e2e	72.69	1	0	2
dd2019040410102595469705a	9822e6b4505c11e998ec7cd30ad32e2e	9.90	0	0	0
dd201904040855247260923f3	17eb26e8512b11e998ec7cd30ad32e2e	34.20	0	0	0
dd20190404155356811396491	b56290c3512611e998ec7cd30ad32e2e	1.00	0	0	0
dd2019040811071774415cd3c	8aa68e0151e811e998ec7cd30ad32e2e	9.90	0	0	0
dd20190413173746480050509	25d50282552a11e998ec7cd30ad32e2e	1.50	0	0	0
dd2019041318233431829fe9e	25d50282552a11e998ec7cd30ad32e2e	8.90	0	0	0
dd20190414115615269162ad6	25d50282552a11e998ec7cd30ad32e2e	37.60	0	0	0
dd201904162102212312123ae	25d50282552a11e998ec7cd30ad32e2e	7.90	0	0	0
dd2019041713172459096e672	d5da2741552611e998ec7cd30ad32e2e	3.96	0	0	0
dd2019041716522249353056f	5d3b50ab601c11e998ec7cd30ad32e2e	0.01	0	0	0
dd2019041722111634321081a	5d3b50ab601c11e998ec7cd30ad32e2e	7.50	0	0	0

grouping(group_id)列为0时，可以看到group_id列都是有值的，为1时则相反，证明当前行是按照group_id来进行分组统计的；

grouping(store_id)同理，为0时store_id列有值，为1时store_id为空，证明当前行时按照store_id来进行分组统计的；

grouping(group_id, store_id)是grouping(group_id)、grouping(store_id)二进制数值组合后转换得到的数字：

- 按照group_id分组，则group_id=0，store_id=1，组合后为01，二进制转换为十进制得到数字1；
- 按照store_id分组，则group_id=1，store_id=0，组合后为10，二进制转换为十进制得到数字2；
- 同时按照group_id和store_id分组，则group_id=0，store_id=0，组合后为00，二进制转换为十进制得到数字0。

因此可以使用grouping操作来判断当前数据是按照哪个字段来分组的。



1.4.1.2 按城市、商圈、店铺分组统计

日期点	订单ID	店铺ID	城市ID	订单金额	商品ID	商品金额
今天	1	超市A	深圳	10元	麻辣花生	3元
	1	超市A	深圳	10元	辣条	2元
	1	超市A	深圳	10元	二锅头	5元
	2	超市A	深圳	15元	圆珠笔	15元

订单量: 2
销售额: 25元

日期点	订单ID	店铺ID	城市ID	订单金额	商品ID	商品金额
今天	1	超市A	深圳	10元	麻辣花生	3元
	2	超市A	深圳	15元	圆珠笔	15元

- 1、按照cn取值，统计的结果没问题；
- 2、判断时，先判断最细粒度的条件，再判断宽粒度的条件；
- 3、可以再加上对应的rn，以便于理解：

row_number() over(partition by s.city_id, o.order_id) city_rn,

row_number() over(partition by s.trade_area_id, o.order_id) trade_area_rn,

row_number() over(partition by o.store_id, o.order_id) store_rn,

```
--insert into yp_dws.dws_sale_daycount
with order_base as
(
    select
--        维度
        SUBSTRING(o.create_date,1,10) as dt,
        province_id, province_name,
        s.city_id,
        s.city_name,
        s.trade_area_id,
        s.trade_area_name,
        s.id as store_id,
        s.store_name,
--        订单量指标
        o.order_id,
--        金额指标
        o.order_amount,
        o.plat_fee,
        o.delivery_fee,
--        判断条件
        o.order_from,
```





```
o.evaluation_id,
o.geval_scores,
o.delievery_id,
o.refund_id,
row_number() over(partition by o.order_id) rn
from yp_dwb.dwb_order_detail o
left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
where o.is_pay=1
-- and o.order_id='dd190404391438bed2'
)
select
-- =====维度=====
dt
,city_id
,city_name
,trade_area_id
,trade_area_name
,store_id
,store_name
,case
    when grouping(store_id)=0
    then 'store'
    when grouping(trade_area_id)=0
    then 'trade_area'
    when grouping(city_id)=0
    then 'city'
    when grouping(dt)=0
    then 'all'
end
as_group_type
-- =====金额=====
-- 销售收入
,sum(coalesce(order_amount, 0)) sale_amt
-- 平台收入
,sum(coalesce(plat_fee,0)) plat_amt
-- 配送成交额
,sum(coalesce(delivery_fee,0)) deliver_sale_amt
-- 小程序成交额
,sum(if(order_from='miniapp', coalesce(order_amount, 0), 0)) mini_app_sale_amt
-- 安卓APP成交额
```



```
,sum(if(order_from='android', coalesce(order_amount, 0), 0)) android_sale_amt
-- 苹果APP成交额
,sum(if(order_from='ios', coalesce(order_amount, 0), 0)) ios_sale_amt
-- PC商城成交额
,sum(if(order_from='pcweb', coalesce(order_amount, 0), 0)) pcweb_sale_amt
-- =====订单量=====
-- 单量
,count(order_id) order_cnt
-- 参评单量
,count(if(evaluation_id is not null, order_id, null)) eva_order_cnt
-- 差评单量
,count(if(evaluation_id is not null and geval_scores<=6, order_id, null)) bad_eva_order_cnt
-- 配送单量
,count(if(delievery_id is not null, order_id, null)) deliver_order_cnt
-- 退款单量
,count(if(refund_id is not null, order_id, null)) refund_order_cnt
-- 小程序单量
,count(if(order_from='miniapp', order_id, null)) miniapp_order_cnt
-- 安卓APP订单量
,count(if(order_from='android', order_id, null)) android_order_cnt
-- 苹果APP订单量
,count(if(order_from='ios', order_id, null)) ios_order_cnt
-- PC商城单量
,count(if(order_from='pcweb', order_id, null)) pcweb_order_cnt
from order_base b
where b.rn=1
group by
grouping sets(
    dt,
    (dt, city_id, city_name, province_id, province_name),
    (dt, trade_area_id, trade_area_name, city_id, city_name, province_id, province_name),
    (dt, store_id, store_name, trade_area_id, trade_area_name, city_id, city_name, province_id,
province_name)
)
--order by dt desc, sum(coalesce(order_amount, 0)) desc
;
```





1.4.1.3 按照品牌-统计

品牌与订单的关系，与商圈和城市的1对多不同，是**多对多**的关系。

因此，按照品牌计算销售额时，不能直接累加整个订单的金额，而是要计算订单中品牌商品的金额，因此**要按照商品goods_id分组**去重。注意此时的订单量统计，需要按照订单id去重。

```
CREATE TABLE yp_dws.dws_sale_daycount(  
  city_id string COMMENT '城市id',  
  city_name string COMMENT '城市name',  
  trade_area_id string COMMENT '商圈id',  
  trade_area_name string COMMENT '商圈名称',  
  store_id string COMMENT '店铺id',  
  store_name string COMMENT '店铺名称',  
  brand_id string COMMENT '品牌id',  
  brand_name string COMMENT '品牌名称',  
  max_class_id string COMMENT '商品大类id',  
  max_class_name string COMMENT '大类名称',  
  mid_class_id string COMMENT '中类id',  
  mid_class_name string COMMENT '中类名称',  
  min_class_id string COMMENT '小类id',  
  min_class_name string COMMENT '小类名称',  
  group_type string COMMENT '分组类型: store, trade_area, city, brand, min_class, mid_c',  
  -----日统计-----  
  -- 销售收入  
  sale_amt DECIMAL(38,2) COMMENT '销售收入',  
  -- 平台收入  
  plat_amt DECIMAL(38,2) COMMENT '平台收入',  
  -- 配送成交额  
  deliver_sale_amt DECIMAL(38,2) COMMENT '配送成交额',  
  -- 小程序成交额  
  mini_app_sale_amt DECIMAL(38,2) COMMENT '小程序成交额',  
  -- 安卓APP成交额  
  android_sale_amt DECIMAL(38,2) COMMENT '安卓APP成交额',  
  -- 苹果APP成交额  
  ios_sale_amt DECIMAL(38,2) COMMENT '苹果APP成交额',  
  -- PC商城成交额  
  pcweb_sale_amt DECIMAL(38,2) COMMENT 'PC商城成交额',  
  -- 成交单量  
  order_cnt BIGINT COMMENT '成交单量',  
  -- 参评单量  
  eva_order_cnt BIGINT COMMENT '参评单量comment=>cmt',  
  -- 差评单量
```

一个店铺下有多个订单;
一个订单只能属于一个店铺

1:n

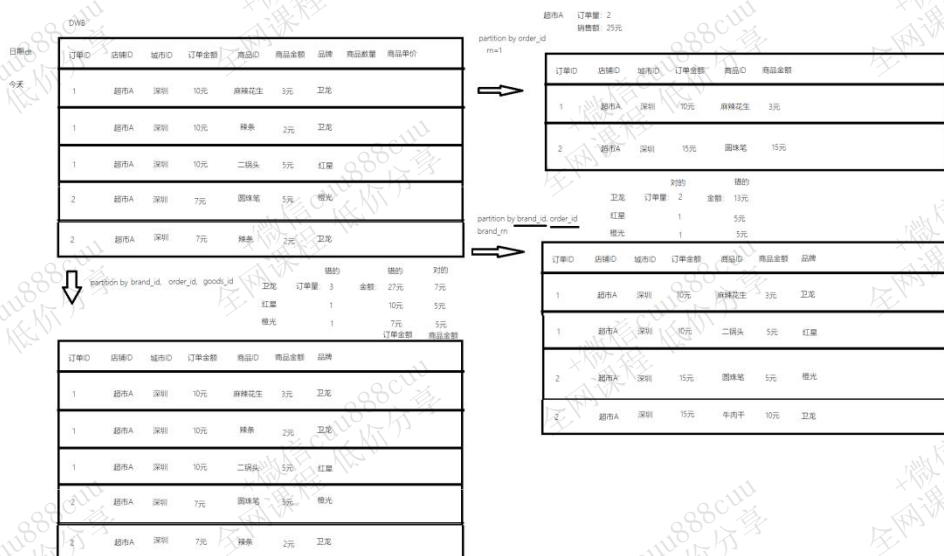
一个品牌有多个订单;
一个订单有多个品牌的商品

n:n

一个小分类有多个订单;
一个订单有多个分类商品

订单





1、品牌的销售额，累加的不是订单的金额，而是商品的金额；因为一个订单下可能有多个品牌的商品；

2、row_number() over(partition by g.brand_id, o.order_id, o.goods_id) brand_goods_rn;

3、rn从where中移除，换到select的条件中，按照不同分组统计时，使用多个不同的rn；

4、判断：

```
case
when grouping(brand_id)=0
then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
when grouping(store_id)=0
then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
when grouping(trade_area_id)=0
then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
when grouping(city_id)=0
then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
when grouping(dt)=0
then sum(if(rn=1, coalesce(order_amount, 0), 0))
end
as sale_amt
```

实现：

```
--insert into yp_dws.dws_sale_daycount
with order_base as
(
select
```





```
-- 维度
SUBSTRING(o.create_date,1,10) as dt,
province_id, province_name,
s.city_id,
s.city_name,
s.trade_area_id,
s.trade_area_name,
s.id as store_id,
s.store_name,
g.brand_id,
g.brand_name,
-- 订单量指标
o.order_id,
-- 金额指标
o.order_amount,
o.total_price as goods_price,
o.plat_fee,
o.delivery_fee,
-- 判断条件
o.order_from,
o.evaluation_id,
o.geval_scores,
o.delievery_id,
o.refund_id,
row_number() over(partition by o.order_id) rn,
row_number() over(partition by s.city_id, o.order_id) city_rn,
row_number() over(partition by s.trade_area_id, o.order_id) trade_area_rn,
row_number() over(partition by o.store_id, o.order_id) store_rn,

row_number() over(partition by g.brand_id, o.order_id, o.goods_id) brand_goods_rn
from yp_dwb.dwb_order_detail o
left join yp_dwb.dwb_shop_details s on o.store_id=s.id
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
where o.is_pay=1
and o.order_id='dd190404391438bed2'
)
select
-- =====维度=====
dt
,city_id
,city_name
```





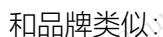
```
,trade_area_id
,trade_area_name
,store_id
,store_name
,brand_id
,brand_name
,case
    when grouping(brand_id)=0
    then 'brand'
    when grouping(store_id)=0
    then 'store'
    when grouping(trade_area_id)=0
    then 'trade_area'
    when grouping(city_id)=0
    then 'city'
    when grouping(dt)=0
    then 'all'
end
as group_type
-- =====金额=====
-- 销售收入
,case
    when grouping(brand_id)=0
    then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(store_id)=0
    then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
    when grouping(trade_area_id)=0
    then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
    when grouping(city_id)=0
    then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
    when grouping(dt)=0
    then sum(if(rn=1, coalesce(order_amount, 0), 0))
end
as sale_amt
-- =====订单量=====
-- 单量
,case
    when grouping(brand_id)=0
    then count(distinct if(brand_goods_rn=1, order_id, null))
    when grouping(store_id)=0
    then count(if(store_rn=1, order_id, null))
```





为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

1.4.1.4 按照种类-统计



课程大全



1、种类(大中小)的销售额，累加的不是订单的金额，而是商品的金额；因为一个订单下可能有多个种类的商品；

2、row_number() over(partition by g.max_class_id, g.mid_class_id, g.min_class_id, o.order_id, o.goods_id) min_class_goods_rn;

3、判断：

```
case
  when grouping(brand_id)=0
  then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(min_class_id)=0
  then sum(if(min_class_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(mid_class_id)=0
  then sum(if(mid_class_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(max_class_id)=0
  then sum(if(max_class_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(store_id)=0
  then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
  when grouping(trade_area_id)=0
  then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
  when grouping(city_id)=0
  then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
  when grouping(dt)=0
  then sum(if(rn=1, coalesce(order_amount, 0), 0))
end
as sale_amt
```

实现：

```
with order_base as
(
  select
  -- 维度
    SUBSTRING(o.create_date,1,10) as dt,
    province_id, province_name,
    s.city_id,
    s.city_name,
    s.trade_area_id,
    s.trade_area_name,
    s.id as store_id,
    s.store_name,
    g.brand_id,
```





```
g.brand_name,
g.max_class_id,
g.max_class_name,
g.mid_class_id,
g.mid_class_name,
g.min_class_id,
g.min_class_name,
-- 订单量指标
o.order_id,
-- 金额指标
o.order_amount,
o.total_price as goods_price,
o.plat_fee,
o.delivery_fee,
-- 判断条件
o.order_from,
o.evaluation_id,
o.geval_scores,
o.delievery_id,
o.refund_id,
row_number() over(partition by o.order_id) rn,
row_number() over(partition by s.city_id, o.order_id) city_rn,
row_number() over(partition by s.trade_area_id, o.order_id) trade_area_rn,
row_number() over(partition by o.store_id, o.order_id) store_rn,
row_number() over(partition by g.brand_id, o.order_id, o.goods_id) brand_goods_rn,
row_number() over(partition by g.max_class_id, o.order_id, o.goods_id)
max_class_goods_rn,
row_number() over(partition by g.max_class_id, g.mid_class_id, o.order_id, o.goods_id)
mid_class_goods_rn,
row_number() over(partition by g.max_class_id, g.mid_class_id, g.min_class_id,
o.order_id, o.goods_id) min_class_goods_rn
from yp_dwb.dwb_order_detail o
left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
where o.is_pay=1
and o.order_id='dd190404391438bed2'
)
select
-- =====维度=====
dt
,city_id
```





```
,city_name
,trade_area_id
,trade_area_name
,store_id
,store_name
,brand_id
,brand_name,
max_class_id,
max_class_name,
mid_class_id,
mid_class_name,
min_class_id,
min_class_name
,case
  when grouping(brand_id)=0
  then 'brand'
  when grouping(min_class_id)=0
  then 'min_class'
  when grouping(mid_class_id)=0
  then 'mid_class'
  when grouping(max_class_id)=0
  then 'max_class'
  when grouping(store_id)=0
  then 'store'
  when grouping(trade_area_id)=0
  then 'trade_area'
  when grouping(city_id)=0
  then 'city'
  when grouping(dt)=0
  then 'all'
end
as group_type
--
=====金额=====
-- 销售收入
,case
  when grouping(brand_id)=0
  then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(min_class_id)=0
  then sum(if(min_class_goods_rn=1, coalesce(goods_price, 0), 0))
  when grouping(mid_class_id)=0
  then sum(if(mid_class_goods_rn=1, coalesce(goods_price, 0), 0))
```





```
when grouping(max_class_id)=0
then sum(if(max_class_goods_rn=1, coalesce(goods_price, 0), 0))
when grouping(store_id)=0
then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
when grouping(trade_area_id)=0
then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
when grouping(city_id)=0
then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
when grouping(dt)=0
then sum(if(rn=1, coalesce(order_amount, 0), 0))
end
as sale_amt,
-- =====订单量=====
-- 单量
,case
when grouping(brand_id)=0
then count(distinct if(brand_goods_rn=1, order_id, null))
when grouping(min_class_id)=0
then count(distinct if(min_class_goods_rn=1, order_id, null))
when grouping(mid_class_id)=0
then count(distinct if(mid_class_goods_rn=1, order_id, null))
when grouping(max_class_id)=0
then count(distinct if(max_class_goods_rn=1, order_id, null))
when grouping(store_id)=0
then count(if(store_rn=1, order_id, null))
when grouping(trade_area_id)=0
then count(if(trade_area_rn=1, order_id, null))
when grouping(city_id)=0
then count(if(city_rn=1, order_id, null))
when grouping(dt)=0
then count(if(rn=1, order_id, null))
end
as order_cnt
-- 参评单量
-- 差评单量
-- 配送单量
-- 退款单量
-- 小程序订单量
-- android订单量
-- ios订单量
```





```
--pc订单量
from order_base o
--where b.rn=1
group by
grouping sets (
    dt,
    (dt, city_id, city_name, province_id, province_name),
    (dt, trade_area_id, trade_area_name, city_id, city_name, province_id, province_name),
    (dt, store_id, store_name, trade_area_id, trade_area_name, city_id, city_name, province_id,
    province_name),
    (dt, brand_id, brand_name),
    (dt, max_class_id, max_class_name),
    (dt, mid_class_id, mid_class_name, max_class_id, max_class_name),
    (dt, min_class_id, min_class_name, mid_class_id, mid_class_name, max_class_id,
    max_class_name)
)
--order by dt desc, sum(coalesce(order_amount, 0)) desc
;
```

1.4.1.5 插入数据

添加insert语句；

数值类型加上非空判断；

把测试用的where条件去掉。

```
insert into yp_dws.dws_sale_daycount
with order_base as
(
    select
--        维度
        SUBSTRING(o.create_date,1,10) as dt,
        s.province_id,
        s.province_name,
        s.city_id,
        s.city_name,
        s.trade_area_id,
        s.trade_area_name,
        s.id as store_id,
        s.store_name,
        g.brand_id,
```





```

        g.brand_name,
        g.max_class_id,
        g.max_class_name,
        g.mid_class_id,
        g.mid_class_name,
        g.min_class_id,
        g.min_class_name,
-- 订单量指标
        o.order_id,
-- 金额指标
        o.order_amount,
        o.total_price as goods_price,
        o.plat_fee,
        o.delivery_fee,
-- 判断条件
        o.order_from,
        o.evaluation_id,
        o.geval_scores,
        o.delievery_id,
        o.refund_id,
        row_number() over(partition by o.order_id) rn,
        row_number() over(partition by s.city_id, o.order_id) city_rn,
        row_number() over(partition by s.trade_area_id, o.order_id) trade_area_rn,
        row_number() over(partition by o.store_id, o.order_id) store_rn,
        row_number() over(partition by g.brand_id, o.order_id, o.goods_id) brand_goods_rn,
        row_number() over(partition by g.max_class_id, o.order_id, o.goods_id)
max_class_goods_rn,
        row_number() over(partition by g.max_class_id, g.mid_class_id, o.order_id, o.goods_id)
mid_class_goods_rn,
        row_number() over(partition by g.max_class_id, g.mid_class_id, g.min_class_id,
o.order_id, o.goods_id) min_class_goods_rn
    from yp_dwb.dwb_order_detail o
        left join yp_dwb.dwb_shop_details s on o.store_id=s.id
        left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
    where o.is_pay=1
-- and o.order_id='dd190404391438bed2'
)
select
-- =====维度=====
-- dt,
    province_id

```





```
,province_name
,city_id
,city_name
,trade_area_id
,trade_area_name
,store_id
,store_name
,brand_id
,brand_name,
max_class_id,
max_class_name,
mid_class_id,
mid_class_name,
min_class_id,
min_class_name
,case
    when grouping(brand_id)=0
    then 'brand'
    when grouping(min_class_id)=0
    then 'min_class'
    when grouping(mid_class_id)=0
    then 'mid_class'
    when grouping(max_class_id)=0
    then 'max_class'
    when grouping(store_id)=0
    then 'store'
    when grouping(trade_area_id)=0
    then 'trade_area'
    when grouping(city_id)=0
    then 'city'
    when grouping(dt)=0
    then 'all'
end
as group_type
-- =====金额=====
-- 销售收入
,case
    when grouping(brand_id)=0
    then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(min_class_id)=0
    then sum(if(min_class_goods_rn=1, coalesce(goods_price, 0), 0))
```





```
when grouping(mid_class_id)=0
then sum(if(mid_class_goods_rn=1, coalesce(goods_price, 0), 0))
when grouping(max_class_id)=0
then sum(if(max_class_goods_rn=1, coalesce(goods_price, 0), 0))
when grouping(store_id)=0
then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
when grouping(trade_area_id)=0
then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
when grouping(city_id)=0
then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
when grouping(dt)=0
then sum(if(rn=1, coalesce(order_amount, 0), 0))
else null
end
as sale_amt,
-- 平台分润金额
case
-- 店铺
when grouping(store_id)=0
then sum(case when rn=1 and store_id is not null then coalesce(plat_fee, 0) else 0 end)
-- 商圈
when grouping(trade_area_id)=0
then sum(case when rn=1 and trade_area_id is not null then coalesce(plat_fee, 0) else
0 end)
-- 城市
when grouping(city_id)=0
then sum(case when rn=1 and city_id is not null then coalesce(plat_fee, 0) else 0 end)
-- 品牌
when grouping(brand_id)=0
then null
-- 小类
when grouping(min_class_id)=0
then null
-- 中类
when grouping(mid_class_id)=0
then null
-- 大类
when grouping(max_class_id)=0
then null
-- 全部
else sum(case when rn=1 then coalesce(plat_fee, 0) else 0 end)
```





```
end
as plat_amt,
-- 配送额
case
-- 店铺
when grouping(store_id)=0
then sum(case when rn=1 and store_id is not null then coalesce(delivery_fee, 0) else 0
end)
-- 商圈
when grouping(trade_area_id)=0
then sum(case when rn=1 and trade_area_id is not null then coalesce(delivery_fee, 0) else
0 end)
-- 城市
when grouping(city_id)=0
then sum(case when rn=1 and city_id is not null then coalesce(delivery_fee, 0) else 0
end)
-- 品牌
when grouping(brand_id)=0
then null
-- 小类
when grouping(min_class_id)=0
then null
-- 中类
when grouping(mid_class_id)=0
then null
-- 大类
when grouping(max_class_id)=0
then null
-- 全部
else sum(case when rn=1 then coalesce(delivery_fee, 0) else 0 end)
end
as deliver_sale_amt,
-- 小程序销售额
case
-- 店铺
when grouping(store_id)=0
then sum(case when o.order_from='miniapp' and rn=1 and store_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 商圈
when grouping(trade_area_id)=0
then sum(case when o.order_from='miniapp' and rn=1 and trade_area_id is not null then
coalesce(order_amount, 0) else 0 end)
```





```
-- 城市
    when grouping(city_id)=0
    then sum(case when o.order_from='miniapp' and rn=1 and city_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 品牌
    when grouping(brand_id)=0
    then sum(case when o.order_from='miniapp' and brand_goods_rn=1 and brand_id is not null
then coalesce(goods_price, 0) else 0 end)
-- 小类
    when grouping(min_class_id)=0
    then sum(case when o.order_from='miniapp' and min_class_goods_rn=1 and min_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 中类
    when grouping(mid_class_id)=0
    then sum(case when o.order_from='miniapp' and mid_class_goods_rn=1 and mid_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 大类
    when grouping(max_class_id)=0
    then sum(case when o.order_from='miniapp' and max_class_goods_rn=1 and max_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 全部
    else sum(case when o.order_from='miniapp' and rn=1 then coalesce(order_amount, 0) else
0 end)
    end
    as mini_app_sale_amt,
-- 安卓销售额
case
-- 店铺
    when grouping(store_id)=0
    then sum(case when o.order_from='android' and rn=1 and store_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 商圈
    when grouping(trade_area_id)=0
    then sum(case when o.order_from='android' and rn=1 and trade_area_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 城市
    when grouping(city_id)=0
    then sum(case when o.order_from='android' and rn=1 and city_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 品牌
    when grouping(brand_id)=0
    then sum(case when o.order_from='android' and brand_goods_rn=1 and brand_id is not null
then coalesce(goods_price, 0) else 0 end)
```





```
-- 小类
when grouping(min_class_id)=0
then sum(case when o.order_from='android' and min_class_goods_rn=1 and min_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 中类
when grouping(mid_class_id)=0
then sum(case when o.order_from='android' and mid_class_goods_rn=1 and mid_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 大类
when grouping(max_class_id)=0
then sum(case when o.order_from='android' and max_class_goods_rn=1 and max_class_id is
not null then coalesce(goods_price, 0) else 0 end)
-- 全部
else sum(case when o.order_from='android' and rn=1 then coalesce(order_amount, 0) else
0 end)
end
as android_sale_amt,
-- ios销售额
case
-- 店铺
when grouping(store_id)=0
then sum(case when o.order_from='ios' and rn=1 and store_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 商圈
when grouping(trade_area_id)=0
then sum(case when o.order_from='ios' and rn=1 and trade_area_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 城市
when grouping(city_id)=0
then sum(case when o.order_from='ios' and rn=1 and city_id is not null then
coalesce(order_amount, 0) else 0 end)
-- 品牌
when grouping(brand_id)=0
then sum(case when o.order_from='ios' and brand_goods_rn=1 and brand_id is not null then
coalesce(goods_price, 0) else 0 end)
-- 小类
when grouping(min_class_id)=0
then sum(case when o.order_from='ios' and min_class_goods_rn=1 and min_class_id is not
null then coalesce(goods_price, 0) else 0 end)
-- 中类
when grouping(mid_class_id)=0
then sum(case when o.order_from='ios' and mid_class_goods_rn=1 and mid_class_id is not
null then coalesce(goods_price, 0) else 0 end)
```



```
--      大类
      when grouping(max_class_id)=0
      then sum(case when o.order_from='ios' and max_class_goods_rn=1 and max_class_id is not
null then coalesce(goods_price, 0) else 0 end)
--      全部
      else sum(case when o.order_from='ios' and rn=1 then coalesce(order_amount, 0) else 0 end)
end
as ios_sale_amt,
-- pc销售额
case
--      店铺
      when grouping(store_id)=0
      then sum(case when o.order_from='pcweb' and rn=1 and store_id is not null then
coalesce(order_amount, 0) else 0 end)
--      商圈
      when grouping(trade_area_id)=0
      then sum(case when o.order_from='pcweb' and rn=1 and trade_area_id is not null then
coalesce(order_amount, 0) else 0 end)
--      城市
      when grouping(city_id)=0
      then sum(case when o.order_from='pcweb' and rn=1 and city_id is not null then
coalesce(order_amount, 0) else 0 end)
--      品牌
      when grouping(brand_id)=0
      then sum(case when o.order_from='pcweb' and brand_goods_rn=1 and brand_id is not null
then coalesce(goods_price, 0) else 0 end)
--      小类
      when grouping(min_class_id)=0
      then sum(case when o.order_from='pcweb' and min_class_goods_rn=1 and min_class_id is not
null then coalesce(goods_price, 0) else 0 end)
--      中类
      when grouping(mid_class_id)=0
      then sum(case when o.order_from='pcweb' and mid_class_goods_rn=1 and mid_class_id is not
null then coalesce(goods_price, 0) else 0 end)
--      大类
      when grouping(max_class_id)=0
      then sum(case when o.order_from='pcweb' and max_class_goods_rn=1 and max_class_id is not
null then coalesce(goods_price, 0) else 0 end)
--      全部
      else sum(case when o.order_from='pcweb' and rn=1 then coalesce(order_amount, 0) else 0
end)
end
as pcweb_sale_amt
```





```
-- =====订单量=====
-- 单量
,case
    when grouping(brand_id)=0
    then count(distinct if(brand_goods_rn=1, order_id, null))
    when grouping(min_class_id)=0
    then count(distinct if(min_class_goods_rn=1, order_id, null))
    when grouping(mid_class_id)=0
    then count(distinct if(mid_class_goods_rn=1, order_id, null))
    when grouping(max_class_id)=0
    then count(distinct if(max_class_goods_rn=1, order_id, null))
    when grouping(store_id)=0
    then count(if(store_rn=1, order_id, null))
    when grouping(trade_area_id)=0
    then count(if(trade_area_rn=1, order_id, null))
    when grouping(city_id)=0
    then count(if(city_rn=1, order_id, null))
    when grouping(dt)=0
    then count(if(rn=1, order_id, null))
end
as_order_cnt
-- 参评单量
,case
    when grouping(brand_id)=0
    then count(distinct if(brand_goods_rn=1 and evaluation_id is not null, order_id, null))
    when grouping(min_class_id)=0
    then count(distinct if(min_class_goods_rn=1 and evaluation_id is not null, order_id,
null))
    when grouping(mid_class_id)=0
    then count(distinct if(mid_class_goods_rn=1 and evaluation_id is not null, order_id,
null))
    when grouping(max_class_id)=0
    then count(distinct if(max_class_goods_rn=1 and evaluation_id is not null, order_id,
null))
    when grouping(store_id)=0
    then count(if(store_rn=1 and evaluation_id is not null, order_id, null))
    when grouping(trade_area_id)=0
    then count(if(trade_area_rn=1 and evaluation_id is not null, order_id, null))
    when grouping(city_id)=0
    then count(if(city_rn=1 and evaluation_id is not null, order_id, null))
    when grouping(dt)=0
    then count(if(rn=1 and evaluation_id is not null, order_id, null))
```





```
end
as eva_order_cnt,
--差评单量
case
-- 店铺
when grouping(store_id)=0
then count(case when evaluation_id is not null and coalesce(geval_scores, 0)<=6 and rn=1
then o.order_id else null end)
-- 商圈
when grouping(trade_area_id)=0
then count(case when evaluation_id is not null and coalesce(geval_scores, 0)<=6 and rn=1
then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when evaluation_id is not null and coalesce(geval_scores, 0)<=6 and rn=1
then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when evaluation_id is not null and coalesce(geval_scores, 0)<=6
and brand_goods_rn=1 then o.order_id else null end)
-- 小类
when grouping(min_class_id)=0
then count(distinct case when evaluation_id is not null and coalesce(geval_scores, 0)<=6
and min_class_goods_rn=1 then o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when evaluation_id is not null and coalesce(geval_scores, 0)<=6
and mid_class_goods_rn=1 then o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when evaluation_id is not null and coalesce(geval_scores, 0)<=6
and max_class_goods_rn=1 then o.order_id else null end)
-- 全部
else count(case when evaluation_id is not null and coalesce(geval_scores, 0)<=6 and rn=1
then o.order_id else null end)
end
as bad_eva_order_cnt,
--配送单量
case
-- 店铺
when grouping(store_id)=0
then count(case when delievery_id is not null and rn=1 then o.order_id else null end)
```





```
-- 商圈
when grouping(trade_area_id)=0
then count(case when delievery_id is not null and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when delievery_id is not null and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when delievery_id is not null and brand_goods_rn=1 then o.order_id
else null end)
-- 小类
when grouping(min_class_id)=0
then count(distinct case when delievery_id is not null and min_class_goods_rn=1 then
o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when delievery_id is not null and mid_class_goods_rn=1 then
o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when delievery_id is not null and max_class_goods_rn=1 then
o.order_id else null end)
-- 全部
else count(case when delievery_id is not null and rn=1 then o.order_id else null end)
end
as deliver_order_cnt,
--退款单量
case
-- 店铺
when grouping(store_id)=0
then count(case when refund_id is not null and rn=1 then o.order_id else null end)
-- 商圈
when grouping(trade_area_id)=0
then count(case when refund_id is not null and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when refund_id is not null and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when refund_id is not null and brand_goods_rn=1 then o.order_id
else null end)
```





```
-- 小类
when grouping(min_class_id)=0
then count(distinct case when refund_id is not null and min_class_goods_rn=1 then
o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when refund_id is not null and mid_class_goods_rn=1 then
o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when refund_id is not null and max_class_goods_rn=1 then
o.order_id else null end)
-- 全部
else count(case when refund_id is not null and rn=1 then o.order_id else null end)
end
as refund_order_cnt,
--小程序订单量
case
-- 店铺
when grouping(store_id)=0
then count(case when o.order_from='miniapp' and rn=1 then o.order_id else null end)
-- 商圈
when grouping(trade_area_id)=0
then count(case when o.order_from='miniapp' and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when o.order_from='miniapp' and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when o.order_from='miniapp' and brand_goods_rn=1 then o.order_id
else null end)
-- 小类
when grouping(min_class_id)=0
then count(distinct case when o.order_from='miniapp' and min_class_goods_rn=1 then
o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when o.order_from='miniapp' and mid_class_goods_rn=1 then
o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when o.order_from='miniapp' and max_class_goods_rn=1 then
```





```
o.order_id else null end)
-- 全部
else count(case when o.order_from='miniapp' and rn=1 then o.order_id else null end)
end
as miniapp_order_cnt,
--android订单量
case
-- 店铺
when grouping(store_id)=0
then count(case when o.order_from='android' and rn=1 then o.order_id else null end)
-- 商圈
when grouping(trade_area_id)=0
then count(case when o.order_from='android' and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when o.order_from='android' and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when o.order_from='android' and brand_goods_rn=1 then o.order_id
else null end)
-- 小类
when grouping(min_class_id)=0
then count(distinct case when o.order_from='android' and min_class_goods_rn=1 then
o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when o.order_from='android' and mid_class_goods_rn=1 then
o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when o.order_from='android' and max_class_goods_rn=1 then
o.order_id else null end)
-- 全部
else count(case when o.order_from='android' and rn=1 then o.order_id else null end)
end
as android_order_cnt,
--ios订单量
case
-- 店铺
when grouping(store_id)=0
then count(case when o.order_from='ios' and rn=1 then o.order_id else null end)
```





```
-- 商圈
when grouping(trade_area_id)=0
then count(case when o.order_from='ios' and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when o.order_from='ios' and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when o.order_from='ios' and brand_goods_rn=1 then o.order_id
else null end)
-- 小类
when grouping(min_class_id)=0
then count(distinct case when o.order_from='ios' and min_class_goods_rn=1 then o.order_id
else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when o.order_from='ios' and mid_class_goods_rn=1 then o.order_id
else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when o.order_from='ios' and max_class_goods_rn=1 then o.order_id
else null end)
-- 全部
else count(case when o.order_from='ios' and rn=1 then o.order_id else null end)
end
as ios_order_cnt,
--pc订单量
case
-- 店铺
when grouping(store_id)=0
then count(case when o.order_from='pcweb' and rn=1 then o.order_id else null end)
-- 商圈
when grouping(trade_area_id)=0
then count(case when o.order_from='pcweb' and rn=1 then o.order_id else null end)
-- 城市
when grouping(city_id)=0
then count(case when o.order_from='pcweb' and rn=1 then o.order_id else null end)
-- 品牌
when grouping(brand_id)=0
then count(distinct case when o.order_from='pcweb' and brand_goods_rn=1 then o.order_id
else null end)
```





```
-- 小类
when grouping(min_class_id)=0
then count(distinct case when o.order_from='pcweb' and min_class_goods_rn=1 then
o.order_id else null end)
-- 中类
when grouping(mid_class_id)=0
then count(distinct case when o.order_from='pcweb' and mid_class_goods_rn=1 then
o.order_id else null end)
-- 大类
when grouping(max_class_id)=0
then count(distinct case when o.order_from='pcweb' and max_class_goods_rn=1 then
o.order_id else null end)
-- 全部
else count(case when o.order_from='pcweb' and rn=1 then o.order_id else null end)
end
as pcweb_order_cnt,
dt
from order_base o
--where b.rn=1
group by
grouping sets(
dt,
(dt, city_id, city_name, province_id, province_name),
(dt, trade_area_id, trade_area_name, city_id, city_name, province_id, province_name),
(dt, store_id, store_name, trade_area_id, trade_area_name, city_id, city_name, province_id,
province_name),
(dt, brand_id, brand_name),
(dt, max_class_id, max_class_name),
(dt, mid_class_id, mid_class_name, max_class_id, max_class_name),
(dt, min_class_id, min_class_name, mid_class_id, mid_class_name, max_class_id,
max_class_name)
)
--order by dt desc, sum(coalesce(order_amount, 0)) desc
;
```

1.5 循环导入

增量计算时，只计算昨天的统计数据：

```
delete from yp_dws.dws_sale_daycount where dt='${TD_DATE}';

with order_base as
```



```
(
    select
    ...
    from yp_dwb.dwb_order_detail o
        left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
        left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
    where o.is_pay=1
--    计算昨天的数据
    and o.dt='${TD_DATE}'
)
select ...
from order_base o
group by
grouping sets(...);
```

2. 商品主题日统计宽表

2.1 建表

[..\第1章 项目介绍与环境准备\报表需求文档.xlsx](#)

主要指标有：下单次数、下单件数、下单金额、被支付次数、被支付件数、被支付金额、被退款次数、被退款件数、被退款金额、被加入购物车次数、被加入购物车件数、被收藏次数、好评数、中评数、差评数。

维度有：商品、日期。

```
drop table if exists yp_dws.dws_sku_daycount;
create table yp_dws.dws_sku_daycount
(
    dt STRING,
    sku_id string comment 'sku_id',
    sku_name string comment '商品名称',
    order_count bigint comment '被下单次数',
    order_num bigint comment '被下单件数',
    order_amount decimal(38,2) comment '被下单金额',
    payment_count bigint comment '被支付次数',
    payment_num bigint comment '被支付件数',
    payment_amount decimal(38,2) comment '被支付金额',
    refund_count bigint comment '被退款次数',
    refund_num bigint comment '被退款件数',
    refund_amount decimal(38,2) comment '被退款金额',
    cart_count bigint comment '被加入购物车次数',
    cart_num bigint comment '被加入购物车件数',
```





```
favor_count bigint comment '被收藏次数',
evaluation_good_count bigint comment '好评数',
evaluation_mid_count bigint comment '中评数',
evaluation_bad_count bigint comment '差评数'
) COMMENT '每日商品行为'
--PARTITIONED BY(dt STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

2.2 导入数据

在DWB层已经将部分数据放在宽表中，宽表通过把相关字段放在同一张表中，可以大大提高数据计算时的效率问题。

2.2.1 问题一

商品的售卖金额、支付金额等，应该使用哪个表的哪个金额字段呢？

2.2.2 问题二

销售额、销售量等指标，还需要对订单使用RN排序去重吗？

2.2.2.1 对比商品销售额

1、RN去重方式

```
with order_sku_rn as
(
    select
        *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail o
    where o.is_valid=1
)
select
    count(o.order_id) order_count,
    sum(coalesce(buy_num,0)) order_num,
```





```
sum(coalesce(total_price,0)) order_amount,
o.goods_id sku_id,
substring(o.create_date, 1, 10) dt
from order_sku rn o
where rn=1
group by substring(o.create_date, 1, 10), o.goods_id
order by order_amount desc;
```

	123 order_count	123 order_num	123 order_amount	sku_id	dt
1	47	103	906.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
2	10	75	742.5	e3e0e15955e811e998ec7cd30ad32e2e	2019-04-16
3	20	25	547.5	c3cefc7d551b11e998ec7cd30ad32e2e	2019-04-21
4	50	50	495	97c77b7655dd11e998ec7cd30ad32e2e	2019-04-04
5	491	491	491	d75c9acc553511e998ec7cd30ad32e2e	2019-04-03
6	30	135	432	a40a85125d0711e998ec7cd30ad32e2e	2019-04-12
7	17	19	416.1	189611ec602311e998ec7cd30ad32e2e	2019-04-21
8	19	33	326.7	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18
9	25	41	323.9	7a75cdd0551b11e998ec7cd30ad32e2e	2019-04-03

2、直接聚合方式

```
select
count(o.order_id) order_count,
sum(coalesce(buy_num,0)) order_num,
sum(coalesce(total_price,0)) order_amount,
o.goods_id sku_id,
substring(o.create_date, 1, 10) dt
from yp_dwb.dwb_order_detail o
where o.is_valid=1
group by substring(o.create_date, 1, 10), o.goods_id
order by order_amount desc;
```

	123 order_count	123 order_num	123 order_amount	sku_id	dt
1	50	108	950.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
2	10	75	742.5	e3e0e15955e811e998ec7cd30ad32e2e	2019-04-16
3	572	572	572	d75c9acc553511e998ec7cd30ad32e2e	2019-04-03
4	20	25	547.5	c3cefc7d551b11e998ec7cd30ad32e2e	2019-04-21
5	52	52	514.8	97c77b7655dd11e998ec7cd30ad32e2e	2019-04-04
6	19	21	459.9	189611ec602311e998ec7cd30ad32e2e	2019-04-21
7	33	141	451.2	a40a85125d0711e998ec7cd30ad32e2e	2019-04-12
8	23	37	366.3	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18
9	25	43	339.7	7a75cdd0551b11e998ec7cd30ad32e2e	2019-04-04

对比结果后，发现两者的数据不同，RN方式商品a0d9e7e5602211e998ec7cd30ad32e2e的销售额为906.4，而直接聚合的销售额为950.4。下钻此商品的订单数据。





2.2.2.2 对比商品的订单列表

1、RN方式

```
with order_sku_rn as
(
    select
        *
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail o
    where o.is_valid=1 and o.goods_id='a0d9e7e5602211e998ec7cd30ad32e2e'
)
select
    o.order_id, o.buy_num, o.total_price, o.goods_id, substring(o.create_date, 1, 10) dt
from order_sku_rn o
where rn=1
order by total_price desc, o.order_id desc;
```

	order_id	buy_num	total_price	goods_id	dt
1	dd1904203592644fc4	10	88	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
2	dd190419382773eeea9	6	59.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-19
3	dd1904183031817715	6	59.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18
4	dd190420394856f225	6	52.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
5	dd190420308691864c	6	52.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
6	dd190521319854885d	5	49.5	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-21
7	dd190420395128bd95	5	44	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
8	dd1904203530974fc4	5	44	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
9	dd190509350722128e	4	39.6	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-09
10	dd190521342282885d	3	29.7	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-21
11	dd1904183430323240	3	29.7	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18

2、直接聚合方式

```
select
    o.order_id, o.buy_num, o.total_price, o.goods_id, substring(o.create_date, 1, 10) dt
-- o.*
from yp_dwb.dwb_order_detail o
where o.is_valid=1 and o.goods_id='a0d9e7e5602211e998ec7cd30ad32e2e'
--and o.order_id='dd190417366979e941'
order by o.total_price desc, o.order_id desc;
```



	order_id	buy_num	total_price	goods_id	dt
1	dd1904203592644fc4	10	88	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
2	dd190419382773ee9	6	59.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-19
3	dd1904183031817715	6	59.4	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18
4	dd190420394856f225	6	52.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
5	dd190420308691864c	6	52.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
6	dd190521319854885d	5	49.5	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-21
7	dd190420395128bd95	5	44	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
8	dd1904203530974fc4	5	44	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-20
9	dd190509350722128e	4	39.6	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-09
10	dd190521342282885d	3	29.7	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-21
11	dd1904183430323240	3	29.7	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18

通过对比发现，订单dd190417366979e941直接聚合的查询结果有两条，比RN方式的结果要多一条。
下钻此订单详情。

22	dd1905173674332d8c	22	dd1905173674332d8c	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-17	
23	dd19050934266381f4	23	dd19050934266381f4	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-05-09	
24	dd1904183949642389	24	dd1904183949642389	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18	
25	dd1904183894503240	25	dd1904183894503240	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18	
26	dd1904183863547715	26	dd1904183863547715	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-18	
27	dd190417384733864c	27	dd190417384733864c	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17	
28	dd190417370780296b	28	dd190417370780296b	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17	
29	dd190417366979e941	29	dd190417366979e941	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17	
30	dd190417366979e941	30	dd190417351878e941	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17	
31	dd190417351878e941	31	dd190417311428296b	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17	
32	dd190417311428296b	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2	19.8	a0d9e7e5602211e998ec7cd30ad32e2e	2019-04-17

2.2.2.3 对比商品订单详情

1、RN方式

```
with order_sku_rn as
(
    select
        *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail o
    where o.is_valid=1 and o.goods_id='a0d9e7e5602211e998ec7cd30ad32e2e'
    and o.order_id='dd190417366979e941'
)
select
-- o.order_id, o.buy_num, o.total_price, o.goods_id, substring(o.create_date, 1, 10) dt
o.*
from order_sku_rn o
where rn=1
order by total_price desc, o.order_id desc;
```





为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

2、直接查询方式

[illegible]

发现此订单，存在重复退款的情况，导致出现多个重复订单。

对于重复退款的订单，实际都是同一个订单，不重复计算，因此需要使用RN去重后聚合。

2.2.3 导入实现



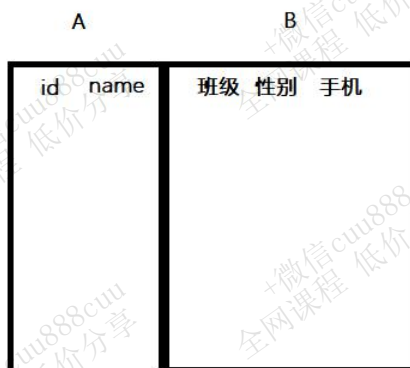


Union 排序去重 所有列都相同，才会去重
Union ALL 简单的拼接 性能高

Full Join

合并行数据：两个表的列数要一致、顺序也要一致

合并列数据



既要合并列，又要合并行

union all
0补齐列
group by
sum

full join
group by
sum

```
insert into hive.yp_dws.dws_sku_daycount
with order_base as (
    select *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
),
-- 下单次数、件数、金额
order_count as (
    select dt, goods_id sku_id, goods_name sku_name,
        count(order_id) order_count,
        sum(coalesce(buy_num,0)) order_num,
        sum(coalesce(total_price,0)) order_amount
    from order_base
    where rn=1
    group by dt, goods_id, goods_name
),
pay_base as(
    select *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
```





```
where is_pay=1
),
-- 支付次数、件数、金额
payment_count as (
    select dt, goods_id sku_id, goods_name sku_name,
           count(order_id) payment_count,
           sum(coalesce(buy_num,0)) payment_num,
           sum(coalesce(total_price,0)) payment_amount
    from pay_base
    where rn=1
    group by dt, goods_id, goods_name
),
refund_base as (
    select *,
           row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
    where refund_id is not null
),
-- 退款次数、件数、金额
refund_count as (
    select dt, goods_id sku_id, goods_name sku_name,
           count(order_id) refund_count,
           sum(coalesce(buy_num,0)) refund_num,
           sum(coalesce(total_price,0)) refund_amount
    from refund_base
    where rn=1
    group by dt, goods_id, goods_name
),
-- 购物车次数、件数
cart_count as (
    select substring(cart.create_time, 1, 10) dt, goods_id sku_id, goods_name sku_name,
           count(cart.id) cart_count,
           sum(coalesce(buy_num,0)) cart_num
    from yp_dwd.fact_shop_cart cart
    left join yp_dwb.dwb_goods_detail g on cart.goods_id=g.id
    where cart.end_date = '9999-99-99'
    group by substring(cart.create_time, 1, 10), goods_id, goods_name
),
-- 收藏次数
favor_count as (
    select substring(c.create_time, 1, 10) dt, goods_id sku_id, goods_name sku_name,
```





```
count(c.id) favor_count
from yp_dwd.fact_goods_collect c
left join yp_dwb.dwb_goods_detail g on c.goods_id=g.id
where c.end_date='9999-99-99'
group by substring(c.create_time, 1, 10), goods_id, goods_name
),
-- 好评、中评、差评数量
evaluation_count as (
select substring(geval_addtime, 1, 10) dt, e.goods_id sku_id, goods_name sku_name,
count(if(geval_scores_goods >= 9, 1, null)) evaluation_good_count,
count(if(geval_scores_goods >6 and geval_scores_goods < 9, 1, null))
evaluation_mid_count,
count(if(geval_scores_goods <= 6, 1, null)) evaluation_bad_count
from yp_dwd.fact_goods_evaluation_detail e
left join yp_dwb.dwb_goods_detail g on e.goods_id=g.id
group by substring(geval_addtime, 1, 10), e.goods_id, goods_name
),
unionall as (
select
dt, sku_id, sku_name,
order_count,
order_num,
order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
0 as cart_count,
0 as cart_num,
0 as favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from order_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
```





```
payment_count,  
payment_num,  
payment_amount,  
0 as refund_count,  
0 as refund_num,  
0 as refund_amount,  
0 as cart_count,  
0 as cart_num,  
0 as favor_count,  
0 as evaluation_good_count,  
0 as evaluation_mid_count,  
0 as evaluation_bad_count
```

```
from payment_count
```

```
union all
```

```
select
```

```
dt, sku_id, sku_name,  
0 order_count,  
0 order_num,  
0 order_amount,  
0 as payment_count,  
0 as payment_num,  
0 as payment_amount,  
refund_count,  
refund_num,  
refund_amount,  
0 as cart_count,  
0 as cart_num,  
0 as favor_count,  
0 as evaluation_good_count,  
0 as evaluation_mid_count,  
0 as evaluation_bad_count
```

```
from refund_count
```

```
union all
```

```
select
```

```
dt, sku_id, sku_name,  
0 order_count,  
0 order_num,  
0 order_amount,  
0 as payment_count,  
0 as payment_num,  
0 as payment_amount,  
0 as refund_count,
```





```
0 as refund_num,
0 as refund_amount,
cart_count,
cart_num,
0 as favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from cart_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
0 as cart_count,
0 as cart_num,
favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from favor_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
0 as cart_count,
0 as cart_num,
```





```
    0 as favor_count,  
    evaluation_good_count,  
    evaluation_mid_count,  
    evaluation_bad_count  
from evaluation_count  
)  
select  
    dt, sku_id, max(sku_name),  
    sum(coalesce(order_count, 0)),  
    sum(coalesce(order_num, 0)),  
    sum(coalesce(order_amount, 0)),  
    sum(coalesce(payment_count, 0)),  
    sum(coalesce(payment_num, 0)),  
    sum(coalesce(payment_amount, 0)),  
    sum(coalesce(refund_count, 0)),  
    sum(coalesce(refund_num, 0)),  
    sum(coalesce(refund_amount, 0)),  
    sum(coalesce(cart_count, 0)),  
    sum(coalesce(cart_num, 0)),  
    sum(coalesce(favor_count, 0)),  
    sum(coalesce(evaluation_good_count, 0)),  
    sum(coalesce(evaluation_mid_count, 0)),  
    sum(coalesce(evaluation_bad_count, 0))  
from unionall  
group by dt, sku_id  
--order by dt, sku_id  
;
```

动态分区插入数据时报错: Query failed (#20210611_024000_00008_i5xqw): Exceeded limit of 100 open writers for partitions/buckets



SQL 错误 [16777237]: Query failed (#20210611_024000_00008_i5xqw): Exceeded limit of 100 open writers for partitions/buckets

[详细信息\(D\) >>](#)

每个presto节点最多只能有100个写入进程，当动态分区过多时，需要增加物理节点的数量，或采取分批执行。测试时将dt字段改为非分区即可。





2.3 循环导入

统计昨日数据：

```
insert into hive.yp_dws.dws_sku_daycount
with order_base as (
    select *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
    where dt='${TD_DATE}'
),
-- 下单次数、件数、金额
order_count as (
    select dt, goods_id sku_id, goods_name sku_name,
        count(order_id) order_count,
        sum(coalesce(buy_num,0)) order_num,
        sum(coalesce(total_price,0)) order_amount
    from order_base
    where rn=1
    group by dt, goods_id, goods_name
),
pay_base as (
    select *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
    where dt='${TD_DATE}' and is_pay=1
),
-- 支付次数、件数、金额
payment_count as (
    select dt, goods_id sku_id, goods_name sku_name,
        count(order_id) payment_count,
        sum(coalesce(buy_num,0)) payment_num,
        sum(coalesce(total_price,0)) payment_amount
    from pay_base
    where rn=1
    group by dt, goods_id, goods_name
),
refund_base as (
    select *,
        row_number() over(partition by order_id, goods_id) rn
    from yp_dwb.dwb_order_detail
```





```
where dt='${TD_DATE}' and refund_id is not null
),
-- 退款次数、件数、金额
refund_count as (
    select dt, goods_id sku_id, goods_name sku_name,
           count(order_id) refund_count,
           sum(coalesce(buy_num,0)) refund_num,
           sum(coalesce(total_price,0)) refund_amount
    from refund_base
    where rn=1
    group by dt, goods_id, goods_name
),
-- 购物车次数、件数
cart_count as (
    select substring(cart.create_time, 1, 10) dt, goods_id sku_id, goods_name sku_name,
           count(cart.id) cart_count,
           sum(coalesce(buy_num,0)) cart_num
    from yp_dwd.fact_shop_cart cart
    left join yp_dwb.dwb_goods_detail g on cart.goods_id=g.id
    where start_date='${TD_DATE}' and end_date = '9999-99-99'
    group by substring(create_time, 1, 10), goods_id
),
-- 收藏次数
favor_count as (
    select substring(c.create_time, 1, 10) dt, goods_id sku_id, goods_name sku_name,
           count(c.id) favor_count
    from yp_dwd.fact_goods_collect c
    left join yp_dwb.dwb_goods_detail g on c.goods_id=g.id
    where start_date='${TD_DATE}' and end_date='9999-99-99'
    group by substring(c.create_time, 1, 10), goods_id
),
-- 好评、中评、差评数量
evaluation_count as (
    select substring(geval_addtime, 1, 10) dt, e.goods_id sku_id, goods_name sku_name,
           count(if(geval_scores_goods >= 9, 1, null)) evaluation_good_count,
           count(if(geval_scores_goods >6 and geval_scores_goods < 9, 1, null))
evaluation_mid_count,
           count(if(geval_scores_goods <= 6, 1, null)) evaluation_bad_count
    from yp_dwd.fact_goods_evaluation_detail e
    left join yp_dwb.dwb_goods_detail g on e.goods_id=g.id
    where start_date='${TD_DATE}'
    group by substring(geval_addtime, 1, 10), e.goods_id
```





```
),  
unionall as (  
    select  
        dt, sku_id, sku_name,  
        order_count,  
        order_num,  
        order_amount,  
        0 as payment_count,  
        0 as payment_num,  
        0 as payment_amount,  
        0 as refund_count,  
        0 as refund_num,  
        0 as refund_amount,  
        0 as cart_count,  
        0 as cart_num,  
        0 as favor_count,  
        0 as evaluation_good_count,  
        0 as evaluation_mid_count,  
        0 as evaluation_bad_count  
    from order_count  
    union all  
    select  
        dt, sku_id, sku_name,  
        0 order_count,  
        0 order_num,  
        0 order_amount,  
        payment_count,  
        payment_num,  
        payment_amount,  
        0 as refund_count,  
        0 as refund_num,  
        0 as refund_amount,  
        0 as cart_count,  
        0 as cart_num,  
        0 as favor_count,  
        0 as evaluation_good_count,  
        0 as evaluation_mid_count,  
        0 as evaluation_bad_count  
    from payment_count  
    union all  
    select  
        dt, sku_id, sku_name,
```





```
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
refund_count,
refund_num,
refund_amount,
0 as cart_count,
0 as cart_num,
0 as favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from refund_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
cart_count,
cart_num,
0 as favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from cart_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
```





```
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
0 as cart_count,
0 as cart_num,
favor_count,
0 as evaluation_good_count,
0 as evaluation_mid_count,
0 as evaluation_bad_count
from favor_count
union all
select
dt, sku_id, sku_name,
0 order_count,
0 order_num,
0 order_amount,
0 as payment_count,
0 as payment_num,
0 as payment_amount,
0 as refund_count,
0 as refund_num,
0 as refund_amount,
0 as cart_count,
0 as cart_num,
0 as favor_count,
evaluation_good_count,
evaluation_mid_count,
evaluation_bad_count
from evaluation_count
)
select
dt, sku_id, max(sku_name),
sum(coalesce(order_count, 0)),
sum(coalesce(order_num, 0)),
sum(coalesce(order_amount, 0)),
sum(coalesce(payment_count, 0)),
sum(coalesce(payment_num, 0)),
sum(coalesce(payment_amount, 0)),
sum(coalesce(refund_count, 0)),
sum(coalesce(refund_num, 0)),
```



```
sum(coalesce(refund_amount,0)),
sum(coalesce(cart_count,0)),
sum(coalesce(cart_num,0)),
sum(coalesce(favor_count,0)),
sum(coalesce(evaluation_good_count,0)),
sum(coalesce(evaluation_mid_count,0)),
sum(coalesce(evaluation_bad_count,0))
from unionall
group by dt, sku_id
--order by dt, sku_id
;
```

3. 用户主题日统计宽表

3.1 建表

[..\第1章 项目介绍与环境准备\报表需求文档.xlsx](#)

主要指标有：登录次数、收藏店铺数、收藏商品数、加入购物车次数、加入购物车金额、下单次数、下单金额、支付次数、支付金额。

维度有：用户、日期。

```
drop table if exists yp_dws.dws_user_daycount;
create table yp_dws.dws_user_daycount
(
    dt STRING,
    user_id string comment '用户 id',
    login_count bigint comment '登录次数',
    store_collect_count bigint comment '店铺收藏数量',
    goods_collect_count bigint comment '商品收藏数量',
    cart_count bigint comment '加入购物车次数',
    cart_amount decimal(38,2) comment '加入购物车金额',
    order_count bigint comment '下单次数',
    order_amount decimal(38,2) comment '下单金额',
    payment_count bigint comment '支付次数',
    payment_amount decimal(38,2) comment '支付金额'
) COMMENT '每日用户行为'
--PARTITIONED BY(dt STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```





3.2 导入数据

思考一下：

- 1、性能是否还能再提高？宽表
- 2、union all合并列是否有替代的办法？

```
insert into yp_dws.dws_user_daycount
-- 登录次数
with login_count as (
    select
        count(id) as login_count,
        login_user as user_id, dt
    from yp_dwd.fact_user_login
    group by login_user, dt
),
-- 店铺收藏数
store_collect_count as (
    select
        count(id) as store_collect_count,
        user_id, substring(create_time, 1, 10) as dt
    from yp_dwd.fact_store_collect
    where end_date='9999-99-99'
    group by user_id, substring(create_time, 1, 10)
),
-- 商品收藏数
goods_collect_count as (
    select
        count(id) as goods_collect_count,
        user_id, substring(create_time, 1, 10) as dt
    from yp_dwd.fact_goods_collect
    where end_date='9999-99-99'
    group by user_id, substring(create_time, 1, 10)
),
-- 加入购物车次数和金额
cart_count_amount as (
    select
        count(cart.id) as cart_count,
        sum(coalesce(g.goods_promotion_price,0)) as cart_amount,
        buyer_id as user_id, substring(cart.create_time, 1, 10) as dt
```





```
from yp_dwd.fact_shop_cart cart, yp_dwb.dwb_goods_detail g
where cart.end_date='9999-99-99' and cart.goods_id=g.id
group by buyer_id, substring(cart.create_time, 1, 10)
),
-- 下单次数和金额
order_count_amount as (
select
count(o.id) as order_count,
sum(coalesce(order_amount,0)) as order_amount,
buyer_id as user_id, substring(create_date, 1, 10) as dt
from yp_dwd.fact_shop_order o, yp_dwd.fact_shop_order_address_detail od
where o.id=od.id
and o.is_valid=1 and o.end_date='9999-99-99' and od.end_date='9999-99-99'
group by buyer_id, substring(create_date, 1, 10)
-- order by buyer_id, substring(create_date, 1, 10)
),
-- 支付次数和金额
payment_count_amount as (
select
count(id) as payment_count,
sum(coalesce(order_amount,0)) as payment_amount,
create_user user_id, substring(create_time, 1, 10) as dt
from yp_dwd.fact_shop_order_address_detail
where is_valid = 1 and pay_time is not null and end_date='9999-99-99'
group by create_user, substring(create_time, 1, 10)
),
groupby as (
select lc.login_count,
0 store_collect_count,
0 goods_collect_count,
0 cart_count, 0 cart_amount,
0 order_count, 0 order_amount,
0 payment_count, 0 payment_amount,
user_id, dt
from login_count lc
union all
select
0 login_count,
scc.store_collect_count,
0 goods_collect_count,
0 cart_count, 0 cart_amount,
```





```
0 order_count, 0 order_amount,
0 payment_count, 0 payment_amount,
user_id, dt
from store_collect_count scc
union all
select
0 login_count,
0 store_collect_count,
gcc.goods_collect_count,
0 cart_count, 0 cart_amount,
0 order_count, 0 order_amount,
0 payment_count, 0 payment_amount,
user_id, dt
from goods_collect_count gcc
union all
select
0 login_count,
0 store_collect_count,
0 goods_collect_count,
cca.cart_count, cart_amount,
0 order_count, 0 order_amount,
0 payment_count, 0 payment_amount,
user_id, dt
from cart_count_amount cca
union all
select
0 login_count,
0 store_collect_count,
0 goods_collect_count,
0 cart_count, 0 cart_amount,
oca.order_count, order_amount,
0 payment_count, 0 payment_amount,
user_id, dt
from order_count_amount oca
union all
select
0 login_count,
0 store_collect_count,
0 goods_collect_count,
0 cart_count, 0 cart_amount,
0 order_count, 0 order_amount,
pca.payment_count, payment_amount,
```





```
        user_id, dt
    from payment_count_amount pca
)
select
    dt,
    user_id,
    -- 登录次数
    sum(coalesce(login_count,0)) login_count,
    -- 店铺收藏数
    sum(coalesce(store_collect_count,0)) store_collect_count,
    -- 商品收藏数
    sum(coalesce(goods_collect_count,0)) goods_collect_count,
    -- 加入购物车次数和金额
    sum(coalesce(cart_count,0)) cart_count,
    sum(coalesce(cart_amount,0)) cart_amount,
    -- 下单次数和金额
    sum(coalesce(order_count,0)) order_count,
    sum(coalesce(order_amount,0)) order_amount,
    -- 支付次数和金额
    sum(coalesce(payment_count,0)) payment_count,
    sum(coalesce(payment_amount,0)) payment_amount
-- , dt
from groupby
group by user_id, dt
;
```

3.3 另一种实现

通过full join代替union all来合并列，并且使用DWB订单宽表来提升性能。

```
insert into yp_dws.dws_user_daycount
-- 登录次数
with login_count as (
    select
        count(id) as login_count,
        login_user as user_id, dt
    from yp_dwd.fact_user_login
    group by login_user, dt
```





```
),
-- 店铺收藏数
store_collect_count as (
    select
        count(id) as store_collect_count,
        user_id, substring(create_time, 1, 10) as dt
    from yp_dwd.fact_store_collect
    where end_date='9999-99-99'
    group by user_id, substring(create_time, 1, 10)
),
-- 商品收藏数
goods_collect_count as (
    select
        count(id) as goods_collect_count,
        user_id, substring(create_time, 1, 10) as dt
    from yp_dwd.fact_goods_collect
    where end_date='9999-99-99'
    group by user_id, substring(create_time, 1, 10)
),
-- 加入购物车次数和金额
cart_count_amount as (
    select
        count(cart.id) as cart_count,
        sum(coalesce(g.goods_promotion_price,0)) as cart_amount,
        buyer_id as user_id, substring(cart.create_time, 1, 10) as dt
    from yp_dwd.fact_shop_cart cart, yp_dwb.dwb_goods_detail g
    where cart.end_date='9999-99-99' and cart.goods_id=g.id
    group by buyer_id, substring(cart.create_time, 1, 10)
),
-- 订单宽表，为后面的下单次数和金额准备数据
order_base as
(
    select
        buyer_id, create_date, order_id, order_amount,
        row_number() over(partition by orderd.order_id) rn
    from yp_dwb.dwb_order_detail orderd
    where orderd.is_valid=1
),
-- 下单次数和金额
order_count_amount as (
    select
```





```
o.buyer_id user_id, substring(o.create_date,1,10) dt,
count(o.order_id) order_count,
sum(coalesce(o.order_amount,0)) order_amount
from order_base o
where o.rn=1
group by o.buyer_id, substring(o.create_date,1,10)
),
-- 支付次数和金额
payment_count_amount as (
select
count(id) as payment_count,
sum(coalesce(order_amount,0)) as payment_amount,
create_user user_id, substring(create_time, 1, 10) as dt
from yp_dwd.fact_shop_order_address_detail
where is_valid = 1 and pay_time is not null and end_date='9999-99-99'
group by create_user, substring(create_time, 1, 10)
),
fulljoin as (
select
coalesce(lc.dt, scc.dt, gcc.dt, cc.dt, oc.dt, pc.dt) dt
,coalesce(lc.user_id, scc.user_id, gcc.user_id, cc.user_id, oc.user_id, pc.user_id)
user_id
,coalesce(login_count,0) login_count
,coalesce(store_collect_count,0) store_collect_count
,coalesce(goods_collect_count,0) goods_collect_count
,coalesce(cart_count,0) cart_count
,coalesce(cart_amount,0) cart_amount
,coalesce(order_count,0) order_count
,coalesce(order_amount,0) order_amount
,coalesce(payment_count,0) payment_count
,coalesce(payment_amount,0) payment_amount
from login_count lc
full join store_collect_count scc on lc.dt=scc.dt and lc.user_id=scc.user_id
full join goods_collect_count gcc on lc.dt=gcc.dt and lc.user_id=gcc.user_id
full join cart_count_amount cc on lc.dt=cc.dt and lc.user_id=cc.user_id
full join order_count_amount oc on lc.dt=oc.dt and lc.user_id=oc.user_id
full join payment_count_amount pc on lc.dt=pc.dt and lc.user_id=pc.user_id
)
select
dt,
user_id,
-- 登录次数
```





```
sum(coalesce(login_count,0)) login_count,
-- 店铺收藏数
sum(coalesce(store_collect_count,0)) store_collect_count,
-- 商品收藏数
sum(coalesce(goods_collect_count,0)) goods_collect_count,
-- 加入购物车次数和金额
sum(coalesce(cart_count,0)) cart_count,
sum(coalesce(cart_amount,0)) cart_amount,
-- 下单次数和金额
sum(coalesce(order_count,0)) order_count,
sum(coalesce(order_amount,0)) order_amount,
-- 支付次数和金额
sum(coalesce(payment_count,0)) payment_count,
sum(coalesce(payment_amount,0)) payment_amount
from fulljoin
group by dt, user_id
-- order by dt, user_id
;
```

3.4 循环导入

统计昨日数据：

```
insert into yp_dws.dws_user_daycount
-- 登录次数
with login_count as (
    select
        count(id) as login_count,
        login_user as user_id, dt
    from yp_dwd.fact_user_login
    where dt='${TD_DATE}'
    group by login_user, dt
),
-- 店铺收藏数
store_collect_count as (
    select
        count(id) as store_collect_count,
        user_id, substring(create_time, 1, 10) as dt
    from yp_dwd.fact_store_collect
    where end_date='9999-99-99' and substring(create_time, 1, 10)='${TD_DATE}'
```





```
group by user_id, substring(create_time, 1, 10)
),
-- 商品收藏数
goods_collect_count as (
select
    count(id) as goods_collect_count,
    user_id, substring(create_time, 1, 10) as dt
from yp_dwd.fact_goods_collect
where end_date='9999-99-99' and substring(create_time, 1, 10)='${TD_DATE}'
group by user_id, substring(create_time, 1, 10)
),
-- 加入购物车次数和金额
cart_count_amount as (
select
    count(cart.id) as cart_count,
    sum(coalesce(g.goods_promotion_price,0)) as cart_amount,
    buyer_id as user_id, substring(cart.create_time, 1, 10) as dt
from yp_dwd.fact_shop_cart cart, yp_dwb.dwb_goods_detail g
where cart.end_date='9999-99-99' and cart.goods_id=g.id and dt='${TD_DATE}'
group by buyer_id, substring(cart.create_time, 1, 10)
),
-- 订单宽表，为后面的下订单次数和金额准备数据
order_base as
(
select
    buyer_id, create_date, order_id, order_amount,
    row_number() over(partition by orderd.order_id) rn
from yp_dwb.dwb_order_detail orderd
where orderd.is_valid=1 and dt='${TD_DATE}'
),
-- 下订单次数和金额
order_count_amount as (
select
    o.buyer_id user_id, substring(o.create_date,1,10) dt,
    count(o.order_id) order_count,
    sum(coalesce(o.order_amount,0)) order_amount
from order_base o
where o.rn=1
group by o.buyer_id, substring(o.create_date,1,10)
),
-- 支付次数和金额
```





```
payment_count_amount as (  
    select  
        count(id) as payment_count,  
        sum(coalesce(order_amount,0)) as payment_amount,  
        create_user user_id, substring(create_time, 1, 10) as dt  
    from yp_dwd.fact_shop_order_address_detail  
    where is_valid=1 and pay_time is not null and end_date='9999-99-99' and start_date='${TD_DATE}'  
    group by create_user, substring(create_time, 1, 10)  
) ,  
fulljoin as (  
    select  
        coalesce(lc.dt, scc.dt, gcc.dt, cc.dt, oc.dt, pc.dt) dt  
        ,coalesce(lc.user_id, scc.user_id, gcc.user_id, cc.user_id, oc.user_id, pc.user_id)  
user_id  
        ,coalesce(login_count,0) login_count  
        ,coalesce(store_collect_count,0) store_collect_count  
        ,coalesce(goods_collect_count,0) goods_collect_count  
        ,coalesce(cart_count,0) cart_count  
        ,coalesce(cart_amount,0) cart_amount  
        ,coalesce(order_count,0) order_count  
        ,coalesce(order_amount,0) order_amount  
        ,coalesce(payment_count,0) payment_count  
        ,coalesce(payment_amount,0) payment_amount  
  
    from login_count lc  
    full join store_collect_count scc on lc.dt=scc.dt and lc.user_id=scc.user_id  
    full join goods_collect_count gcc on lc.dt=gcc.dt and lc.user_id=gcc.user_id  
    full join cart_count_amount cc on lc.dt=cc.dt and lc.user_id=cc.user_id  
    full join order_count_amount oc on lc.dt=oc.dt and lc.user_id=oc.user_id  
    full join payment_count_amount pc on lc.dt=pc.dt and lc.user_id=pc.user_id  
)  
select  
    dt,  
    user_id,  
    -- 登录次数  
    sum(coalesce(login_count,0)) login_count,  
    -- 店铺收藏数  
    sum(coalesce(store_collect_count,0)) store_collect_count,  
    -- 商品收藏数  
    sum(coalesce(goods_collect_count,0)) goods_collect_count,  
    -- 加入购物车次数和金额  
    sum(coalesce(cart_count,0)) cart_count,  
    sum(coalesce(cart_amount,0)) cart_amount,
```





```
-- 下单次数和金额
sum(coalesce(order_count,0)) order_count,
sum(coalesce(order_amount,0)) order_amount,
-- 支付次数和金额
sum(coalesce(payment_count,0)) payment_count,
sum(coalesce(payment_amount,0)) payment_amount
from fulljoin
group by dt, user_id
-- order by dt, user_id
;
```

